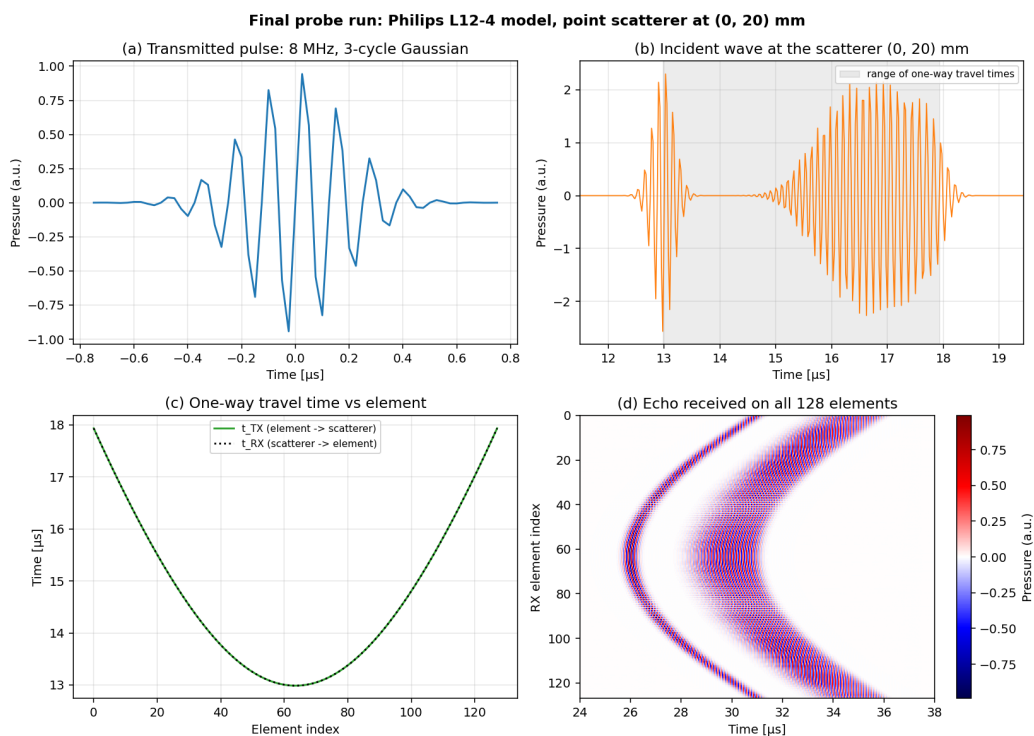


Team 3: Acoustic Propagation

How the sound pulse travels from the probe to a scatterer and back

NITK-UsoundSim · 2-D ultrasound simulator · explanation, input, output and run guide

- The team's code sends a short ultrasound pulse from every element of a linear array, adds up what arrives at a point scatterer, and then sends the echo back to every element.
- It works with a mechanical **pressure wave**: distance, travel time ($t = d / c$) and attenuation (dB/MHz/cm) are the three key ideas.
- One command, `python3 run_team3.py`, runs the 22 unit tests (all pass), the team's own demo (64 elements, 5 MHz) and a second run with the project's final probe (Philips L12-4 model: 128 elements, 0.30 mm pitch, 8 MHz).
- Measured run time on the test Mac: about 2 to 5 seconds.



Final probe run (128 elements, 8 MHz, scatterer at (0, 20) mm). Produced by `run_team3.py`.

Contents

1 Overview	3
Files in this folder	3
2 Theory	4
2.1 Sound is a pressure wave	4
2.2 Speed of sound, wavelength and pitch	4
2.3 Time of flight: $t = d / c$	4
2.4 Attenuation in dB/MHz/cm	4
2.5 The transmitted pulse	5
2.6 Forward propagation: a sum over elements	5
2.7 Return propagation: shifting a sampled signal	5
2.8 Linear superposition	5
2.9 Sampling and Nyquist	5
2.10 Simplifications of this model	5
3 Code explanation	6
3.1 config.py: the single source of truth	6
3.2 acoustic_propagation.py: function by function	6
3.3 demo_acoustic_propagation.py	7
3.4 test_acoustic_propagation.py (22 tests)	7
3.5 check_arr.py and __init__.py	7
3.6 run_team3.py (added for this demo)	8
3.7 Full code listings	8
4 Input	28
5 Output	29
5.1 Unit tests	29
5.2 Team demo, Figure 1: geometry, pulse, delay, attenuation	29
5.3 Team demo, Figure 2: signals through the pipeline	30
5.4 Final probe run (Philips L12-4 model)	31
5.5 Attenuation at 4, 8 and 12 MHz	32
5.6 Console output	32
6 How to run	35
Running the team's own commands	35
Troubleshooting	35
7 Check yourself	36

1 Overview

What Team 3 does and what is in the folder

Team 3 builds the **acoustic propagation** part of the NITK-UsoundSim ultrasound simulator. Its job is to answer one question: *when every element of the probe sends a short pulse, what pressure arrives at a point in the tissue, and what pressure comes back to each element?*

The code follows this chain:

- **TX elements** (the array) send a Gaussian pulse.
- **forward_propagation()**: every pulse travels to the scatterer; the pulses are delayed, weakened (attenuated) and added together. This gives the *incident wave* at the scatterer.
- **mock_tissue_interaction()**: a simple stand-in for the tissue: the wave is multiplied by the scatterer's amplitude to give the *reflected wave*.
- **return_propagation()**: the reflected wave travels back to each receive element, delayed and attenuated again. The result is `pressure[element, time]`.

Place in the pipeline

Team 3 sits between transmit beamforming (Team 2) and tissue interaction (Team 4) / receive beamforming (Team 5): it moves the pressure wave from the probe into the tissue and back.

Files in this folder

File / folder	What it is
README.md	Short step-by-step run guide.
requirements.txt	Python packages needed: numpy, matplotlib.
run_team3.py	The one command to run. Added for this demo; it only calls the team's functions.
code/nitk_usoundsim/	The team's ORIGINAL package, copied unchanged (byte-identical).
__init__.py	Marks the folder as a Python package (one docstring line).
config.py	All constants: speed of sound, frequency, sampling rate, array size, phantom points.
acoustic_propagation.py	The main module: pulse, geometry, delays, attenuation, forward and return propagation.
demo_acoustic_propagation.py	The team's demo: makes two figures and two .npz data files.
test_acoustic_propagation.py	22 unit tests.
check_arr.py	Tiny helper that loads a saved .npz file and prints its contents.
requirements.txt	The team's own list of packages.
*.npz, *.png	Outputs the team saved earlier (kept for reference; the new run writes to output/).
input/team3_run_parameters.json	Sample input: array, pulse and scatterer settings for both runs.
output/	Everything made by run_team3.py (plots, data, console log, screenshot).
Team_3_Acoustic_Propagation_Explana tion.pdf	This document.

2 Theory

The physics behind the code, in simple words

2.1 Sound is a pressure wave

Ultrasound is a mechanical wave: the tissue is squeezed and stretched, so the local pressure goes up and down. The probe element pushes the tissue, the push travels outwards, and later the echo pushes back on the element. Team 3's code works only with this **mechanical pressure** (in arbitrary units, a.u.); turning pressure into an electrical signal is done by other teams.

2.2 Speed of sound, wavelength and pitch

In soft tissue sound travels at about **c = 1540 m/s**. The wavelength is the distance the wave moves in one period of the carrier:

$$\lambda = \frac{c}{f_0} = \frac{1540}{5 \times 10^6} = 0.308 \text{ mm}$$

Team design: $f_0 = 5 \text{ MHz}$. The element pitch is $\lambda/2 = 0.154 \text{ mm}$ (config.py).

A linear array has N elements placed in a straight line, one pitch apart, centred on $x = 0$:

$$x_n = \left(n - \frac{N-1}{2}\right) \cdot \text{pitch}, \quad n = 0, 1, \dots, N-1$$

2.3 Time of flight: $t = d / c$

The element at $(x_n, 0)$ and a scatterer at (x_s, z_s) form a right triangle, so the straight-line distance comes from Pythagoras. Dividing by c gives the travel time:

$$d_n = \sqrt{(x_s - x_n)^2 + z_s^2}, \quad t_n = \frac{d_n}{c}$$

Example (checked by the run)

For the scatterer at (0, 20) mm the centre element is 20 mm away, so $t = 0.020 / 1540 = 12.99 \text{ us}$. With the 128-element L12-4 probe the edge element ($x = 19.05 \text{ mm}$) is 27.62 mm away, so $t = 17.94 \text{ us}$. The echo needs the same time again to come back: round trip = $2z/c = 25.97 \text{ us}$ on axis.

2.4 Attenuation in dB/MHz/cm

Tissue absorbs and scatters energy, so the wave gets weaker with distance, and faster at high frequency. The model uses the usual soft-tissue rule with $\alpha_0 = 0.5 \text{ dB/MHz/cm}$:

$$A_{dB} = \alpha_0 \cdot f_{0, \text{MHz}} \cdot d_{\text{cm}}, \quad A_{\text{lin}} = 10^{-A_{dB}/20}$$

Why divide by **20** and not 10? The decibel is defined on *power*: $\text{dB} = 10 \log_{10}(\text{P ratio})$. Power is proportional to pressure squared, so for a pressure *amplitude* ratio $\text{dB} = 10 \log_{10}(p^2 \text{ ratio}) = 20 \log_{10}(p \text{ ratio})$. The code multiplies pressure amplitudes, so it must use /20.

Example

5 MHz, 2 cm: $A = 0.5 \times 5 \times 2 = 5 \text{ dB}$, so the amplitude factor is $10^{(-5/20)} = 0.562$ one way. There and back: 10 dB, factor 0.316 (these are exactly the numbers the run prints for $z = 20 \text{ mm}$).

2.5 The transmitted pulse

Each element sends a short burst: a sine wave at the centre frequency f_0 , shaped by a bell-shaped (Gaussian) envelope so that it starts and ends smoothly:

$$p(t) = \sin(2\pi f_0 t) \exp\left(-\frac{t^2}{2\sigma^2}\right), \quad \sigma = \frac{N_{\text{cycles}}}{2f_0}$$

With $N_{\text{cycles}} = 3$: at 5 MHz $\sigma = 0.3$ us; at 8 MHz $\sigma = 0.1875$ us. `generate_pulse()` samples the pulse from -4 sigma to $+4$ sigma (the envelope is about e^{-8} there, practically zero). A shorter pulse gives better axial (depth) resolution.

2.6 Forward propagation: a sum over elements

All elements fire together (no focusing: the extra transmit delay τ_{TX} is zero). The pressure at the scatterer is the **sum** of all the delayed, attenuated pulses:

$$p_{\text{inc}}(t) = \sum_{n=0}^{N-1} A_{TX}(n) p(t - \tau_{TX}(n) - t_{TX}(n))$$

Because $p(t)$ is a formula, the code evaluates it exactly at the shifted time; no interpolation is needed.

2.7 Return propagation: shifting a sampled signal

The reflected wave is now a list of numbers (samples), not a formula. To delay it by $t_{RX}(m)$ for receive element m , the code reads the reflected wave at the times $t - t_{RX}(m)$ using **linear interpolation** (`np.interp`): between two samples it draws a straight line. Times before the start or after the end are set to 0.

$$p_{RX,m}(t) = A_{RX}(m) p_{\text{refl}}(t - t_{RX}(m))$$

2.8 Linear superposition

The model is linear: waves simply add. With several scatterers, each echo is computed on its own and the received signals are added. A unit test checks that the total equals the sum of the parts.

2.9 Sampling and Nyquist

Signals are stored at $f_s = 40$ MHz, one sample every 25 ns. The Nyquist rule says f_s must be more than twice the highest frequency present. For a 5 MHz or 8 MHz pulse (bandwidth reaching roughly 12 MHz at 8 MHz) 40 MHz is enough, giving 8 (5 MHz) or 5 (8 MHz) samples per carrier period.

2.10 Simplifications of this model

- Sound goes in **straight lines** at a **constant speed** (no refraction, one tissue type).
- Each element is a **point**: no element width, no directivity, no apodization.
- **No 1/r spreading**: only attenuation reduces the amplitude, not the geometric spreading of the wave.
- The tissue is a **placeholder**: `mock_tissue_interaction()` just multiplies by 'amp'. No impedance, reflection coefficient or speckle.
- Attenuation is applied at the centre frequency only (no change of pulse shape with depth).
- No transmit focusing by default ($\tau_{tx} = 0$), linear physics only.

3 Code explanation

Every file, section by section

3.1 config.py: the single source of truth

config.py holds all constants that other modules must import instead of re-typing them:

Name	Value	Meaning
C	1540.0 m/s	speed of sound in tissue
ALPHA_0	0.5 dB/MHz/cm	attenuation coefficient
F0	5.0e6 Hz	centre frequency
FS	40.0e6 Hz	sampling frequency
N_ELEMENTS	64	number of array elements
LAMBDA, PITCH	$C/F0 = 0.308$ mm, $LAMBDA/2 = 0.154$ mm	wavelength and element spacing
X_MIN..X_MAX, Z_MIN..Z_MAX	-10..10 mm, 0..40 mm	imaging field of view
N_SCANLINES	64	number of image lines (used by later teams)
PHANTOM_POINTS	(0,10), (0,20), (0,30) mm, amp 1.0	three test scatterers
DYNAMIC_RANGE_DB, OUTPUT_SIZE	50 dB, (512, 512)	display settings (used by later teams)

3.2 acoustic_propagation.py: function by function

The top docstring describes the signal flow and units (metres, seconds, Hz). All constants are imported from config.py. $N_CYCLES = 3$ is the one design choice about pulse length.

Function	What it does, in plain words
<code>_pulse_shape(t, f0, n_cycles)</code>	The pulse formula $p(t) = \sin(2 \pi f_0 t) \exp(-t^2 / 2 \sigma^2)$ with $\sigma = n_cycles / (2 f_0)$. It works for any time value, so a delayed pulse is just <code>_pulse_shape(t - delay)</code> . The leading underscore means it is an internal helper.
<code>generate_pulse(fs, f0, n_cycles)</code>	Makes a sampled copy of the pulse for plotting: a time axis from -4σ to $+4 \sigma$ in steps of $1/fs$, and $p(t)$ on it. Returns (t, p). Propagation does not use this array; it uses <code>_pulse_shape</code> directly.
<code>create_linear_array(n_elements, pitch)</code>	Returns the x-position of every element: $(index - (N-1)/2) \times pitch$, so the array is centred on 0. Element 0 is at the most negative x.
<code>calculate_tx_distance(element_x, x_s, z_s)</code>	Pythagoras distance $\sqrt{(x_s - x_n)^2 + z_s^2}$ from each transmit element to the scatterer.
<code>calculate_rx_distance(element_x, x_s, z_s)</code>	The same formula for the way back (scatterer to each receive element). It is kept as a separate function so receive-only effects can be added later without touching the transmit path.
<code>calculate_propagation_delay(distance, c)</code>	Travel time $t = d / c$.
<code>calculate_attenuation(distance, f0, alpha0)</code>	Converts Hz to MHz and m to cm, computes the loss in dB = $\alpha_0 \times f_MHz \times d_cm$ and returns the linear amplitude factor $10^{(-dB/20)}$, a number between 0 and 1.

Function	What it does, in plain words
<code>forward_propagation(scatterer, element_x, t_axis, ...)</code>	Computes distances, times and attenuation for every element, sets <code>tau_tx</code> to zeros if not given (a hook for a future transmit-focusing team), and then loops over the elements adding $A_{TX}(n) \times p(t - \tau_{tx}(n) - t_{TX}(n))$. Returns (<code>incident_wave</code> , <code>d_tx</code> , <code>t_tx</code>).
<code>mock_tissue_interaction(incident_wave, scatterer)</code>	Temporary placeholder for Team 4: returns <code>incident_wave</code> x <code>scatterer['amp']</code> . Same length in and out, so the real tissue module can be plugged in later without changing <code>return_propagation()</code> .
<code>return_propagation(reflected_wave, t_axis, scatterer, element_x, ...)</code>	For every receive element <code>m</code> it computes <code>d_RX</code> , <code>t_RX</code> and <code>A_RX</code> , then shifts the sampled reflected wave with <code>np.interp(t_axis - t_RX(m), t_axis, reflected_wave, left=0, right=0)</code> and multiplies by <code>A_RX(m)</code> . Returns (<code>pressure[element, time]</code> , <code>d_rx</code> , <code>t_rx</code>).
<code>simulate_acoustic_propagation(phantom_points, ...)</code>	The whole pipeline. (1) builds the array and pulse; (2) finds the longest round trip over all scatterers and builds a shared time axis from minus the pulse half-width to that time plus a margin (15 % of the round trip + half-width); (3) for each scatterer runs forward -> mock tissue -> return and adds the result to the total (linear superposition); (4) returns a dictionary with <code>t_axis</code> , <code>pressure</code> (and its alias <code>received_pressure</code>), <code>incident_pressure</code> , <code>reflected_pressure</code> , <code>element_x</code> , the pulse and a <code>per_scatterer</code> list with all the delays and partial signals.

3.3 demo_acoustic_propagation.py

- Imports `config` and `acoustic_propagation` (with a fall-back import if run from inside the package).
- OUT_DIR** is the folder of the demo file itself: by default the demo saves next to its own source. `run_team3.py` instead passes `out_dir=output/` so the original folder is not touched.
- make_figures(out_dir)** runs `simulate_acoustic_propagation()` and draws Figure 1 (array + scatterers, pulse, delay vs depth, attenuation vs depth) and Figure 2 (incident waves, reflected waves, the waveform on RX element #32, and the element-by-time image).
- save_part_outputs(result, out_dir)** saves two self-describing .npz files: PART 1 (forward propagation: incident pressure at each scatterer) and PART 2 (return propagation: received pressure on all 64 elements). Every array has labels and units stored next to it.
- The `__main__` block calls both functions and prints the file names and array shapes.

3.4 test_acoustic_propagation.py (22 tests)

Test class	What it checks
<code>TestArrayGeometry</code> (4)	element count, spacing = pitch, array centred on 0, $LAMBDA = C/F0$ and $PITCH = LAMBDA/2$
<code>TestDistancesAndDelays</code> (7)	TX and RX distance formulas, on-axis distance = depth, $t = d/c$ for TX and RX, deeper = later, off-axis target gives different delays
<code>TestAttenuation</code> (4)	matches the dB formula, 20 mm at 5 MHz = 5 dB, deeper = weaker, factor in (0, 1]
<code>TestPulse</code> (2)	pulse sampled at $1/FS$, peak near $t = 0$
<code>TestFullPipeline</code> (5)	output shape, real and finite, 20 mm round trip = 25.97 us (50 ns tolerance), deeper points later, superposition

3.5 check_arr.py and __init__.py

`check_arr.py` is a small scratch script: it loads

`nitk_usoundsim/part2_return_propagation_outputs.npz` (path relative to the code/ folder) and prints the description, the reflected pressure array, its shape and row labels. Two other loads are commented out. `run_team3.py` does not call it. `__init__.py` contains only a docstring.

3.6 run_team3.py (added for this demo)

- Adds code/ to the Python path, reads input/team3_run_parameters.json and copies all printed text into a log.
- Step 1: runs the unit tests in a separate process inside code/ and counts the passes.
- Step 2: calls the demo's make_figures() and save_part_outputs() with out_dir = output/.
- Step 3: final probe run by arguments only: create_linear_array(128, 0.30e-3), generate_pulse(f0=8e6), a 40 MHz time axis from minus the pulse half-width, forward_propagation(..., f0=8e6), mock_tissue_interaction, return_propagation(..., f0=8e6); prints key numbers, saves a figure and an .npz.
- Step 4: calculate_attenuation at 4, 8 and 12 MHz vs depth.
- Finally writes console_log.txt and a screenshot-style console_screenshot.png.

3.7 Full code listings

code/nitk_usoundsim/__init__.py

```
1 """NITK-UsoundSim - Acoustic Propagation module (Phase 1)."""
```

code/nitk_usoundsim/config.py

```
1 """
2 config.py
3 =====
4
5 Global configuration for NITK-UsoundSim.
6
7 This is the SINGLE SOURCE OF TRUTH for medium, transducer, imaging-FOV,
8 phantom and display parameters. Every other module (acoustic propagation,
9 tissue interaction, beamforming, image formation, ...) MUST import these
10 values from here rather than redefining or hard-coding them.
11 """
12
13 import numpy as np
14
15 # -----
16 # Medium parameters
17 # -----
18 C = 1540.0          # speed of sound in tissue [m/s]
19 ALPHA_0 = 0.5       # attenuation coefficient [dB / MHz / cm]
20
21 # -----
22 # Transducer array
23 # -----
24 F0 = 5.0e6          # transducer centre frequency [Hz]
25 FS = 40.0e6         # sampling frequency [Hz]
26 N_ELEMENTS = 64     # number of array elements
27 LAMBDA = C / F0     # wavelength in tissue at F0 [m]
28 PITCH = LAMBDA / 2  # element pitch (lambda/2 spacing) [m]
29
30 # -----
31 # Imaging field of view
32 # -----
33 X_MIN, X_MAX = -0.010, 0.010  # lateral FOV [m]
34 Z_MIN, Z_MAX = 0.000, 0.040  # axial (depth) FOV [m]
35 N_SCANLINES = 64
36
37 # -----
38 # Calibration targets (synthetic point phantom)
39 # -----
40 PHANTOM_POINTS = [
41     {"x": 0.000, "z": 0.010, "amp": 1.0},
42     {"x": 0.000, "z": 0.020, "amp": 1.0},
43     {"x": 0.000, "z": 0.030, "amp": 1.0},
44 ]
45
46 DYNAMIC_RANGE_DB = 50.0
47 OUTPUT_SIZE = (512, 512)
```

code/nitk_usoundsim/acoustic_propagation.py

```

1  """
2  acoustic_propagation.py
3  =====
4
5  Acoustic Propagation module for NITK-UsoundSim (Phase 1).
6
7  Implements a SIMPLE, FIELD-II-STYLE LINEAR SPATIAL IMPULSE RESPONSE MODEL
8  for a 64-element linear array. This is an educational simplification of
9  the real spatial-impulse-response formalism used by Field II, not a
10 full acoustic-field simulator (no element directivity/apodization,
11 no diffraction-limited beam profile, no nonlinear/pseudospectral
12 propagation). Those belong to a later, more realistic propagation model.
13
14 Signal flow (this module owns everything from the TX pulse up to the
15 mechanical pressure arriving back at the 64 receive elements)::
16
17     64 TX elements
18     |
19     forward_propagation()      -> incident mechanical wave at scatterer
20     |
21     mock_tissue_interaction()  -> reflected mechanical wave (TEMPORARY,
22     |                               swappable stand-in for the real
23     |                               Tissue Interaction module)
24     return_propagation()      -> pressure[64 RX elements, time]
25
26 The data stays a MECHANICAL ACOUSTIC PRESSURE WAVEFORM throughout - this
27 module never touches RF/electrical channel data (that is a later stage,
28 e.g. transducer receive sensitivity + electronics, owned by another
29 module).
30
31 Units
32 -----
33 - Distances: metres.
34 - Time: seconds.
35 - Frequency: Hz internally; converted to MHz only where ALPHA_0's
36   dB/MHz/cm convention requires it.
37 - ALPHA_0 is a dB/MHz/cm *amplitude* attenuation coefficient (see
38   calculate_attenuation for the exact dB -> linear-amplitude conversion).
39
40 All physical constants come from ``nitk_usoundsim.config`` and are never
41 redefined here.
42 """
43
44 from typing import Dict, List, Optional, Tuple
45
46 import numpy as np
47
48 from nitk_usoundsim.config import (
49     ALPHA_0,
50     C,
51     F0,
52     FS,
53     N_ELEMENTS,
54     PHANTOM_POINTS,
55     PITCH,
56 )
57
58 # -----
59 # Assumption: transmitted pulse shape
60 # -----
61 # The project brief leaves the exact pulse duration/envelope to be chosen
62 # reasonably. We use a Gaussian-modulated sinusoid
63 #
64 #  $p(t) = \sin(2\pi F_0 t) * \exp(-t^2 / (2\sigma^2))$ 
65 #
66 # with sigma set so the envelope spans approximately N_CYCLES periods of
67 # the F0 carrier ( $\sigma = N\_CYCLES / (2 * F_0)$ ). N_CYCLES = 3 is a typical
68 # choice for a diagnostic-imaging-like transmit pulse: short enough for
69 # reasonable axial resolution, long enough to have a well-defined centre
70 # frequency. This is documented here as the single assumption governing
71 # pulse shape; change N_CYCLES to explore other pulse lengths.
72 N_CYCLES = 3
73
74

```

```

75 def _pulse_shape(t: np.ndarray, f0: float = F0, n_cycles: int = N_CYCLES) -> np.ndarray:
76     """
77     Closed-form Gaussian-modulated sinusoidal pulse, centred at t = 0.
78
79     Physical meaning: the mechanical pressure waveform emitted by a
80     transducer element, evaluated at arbitrary time(s) ``t``. Because
81     this is a closed-form function of time (not a fixed sample array),
82     it can be evaluated directly at any propagation-delayed time
83     (t - t_TX(n)) without needing to resample/interpolate – this keeps
84     forward propagation numerically exact regardless of delay size.
85
86     Parameters
87     -----
88     t : array_like
89         Time(s) [s] at which to evaluate the pulse, relative to the
90         pulse centre (t=0 is the envelope peak).
91     f0 : float
92         Carrier (centre) frequency [Hz].
93     n_cycles : int
94         Approximate number of carrier cycles under the Gaussian envelope.
95
96     Returns
97     -----
98     np.ndarray
99         Pressure amplitude (arbitrary units, peak ~ 1) at each time in t.
100    """
101    sigma = n_cycles / (2.0 * f0)
102    envelope = np.exp(-(t ** 2) / (2.0 * sigma ** 2))
103    return np.sin(2.0 * np.pi * f0 * t) * envelope
104
105
106 def generate_pulse(
107     fs: float = FS, f0: float = F0, n_cycles: int = N_CYCLES
108 ) -> Tuple[np.ndarray, np.ndarray]:
109     """
110     Generate a short, sampled Gaussian-modulated sinusoidal TX pulse for
111     display/reference purposes (e.g. plotting "the transmitted pulse").
112
113     Internally, forward/return propagation do NOT resample this array;
114     they re-evaluate ``_pulse_shape`` directly at whatever delayed time
115     is needed. This function exists to give a concrete, sampled
116     ``p(t)`` waveform matching the brief's example::
117
118         p(t) = sin(2*pi*f0*t) * Gaussian_envelope(t)
119
120     Parameters
121     -----
122     fs : float
123         Sampling frequency [Hz] (defaults to the global FS).
124     f0 : float
125         Carrier frequency [Hz] (defaults to the global F0).
126     n_cycles : int
127         Number of carrier cycles under the envelope (see N_CYCLES).
128
129     Returns
130     -----
131     t : np.ndarray
132         Time axis [s], centred at 0.
133     p : np.ndarray
134         Sampled pulse amplitude at each time in ``t``.
135     """
136     sigma = n_cycles / (2.0 * f0)
137     t_half_width = 4.0 * sigma # generous support: envelope is ~e^-8 there
138     t = np.arange(-t_half_width, t_half_width, 1.0 / fs)
139     p = _pulse_shape(t, f0=f0, n_cycles=n_cycles)
140     return t, p
141
142
143 def create_linear_array(
144     n_elements: int = N_ELEMENTS, pitch: float = PITCH
145 ) -> np.ndarray:
146     """
147     Compute the lateral (x) position of every element of a linear array,
148     centred on x = 0 (the array's geometric centre defines the imaging
149     origin, consistent with the symmetric X_MIN/X_MAX FOV in config.py).
150
151     Parameters

```

```

152     -----
153     n_elements : int
154         Number of elements (defaults to the global N_ELEMENTS).
155     pitch : float
156         Centre-to-centre element spacing [m] (defaults to the global PITCH).
157
158     Returns
159     -----
160     np.ndarray, shape (n_elements,)
161         x-position [m] of each element, element 0 at the most negative x.
162     """
163     indices = np.arange(n_elements) - (n_elements - 1) / 2.0
164     return indices * pitch
165
166
167 def calculate_tx_distance(
168     element_x: np.ndarray, x_s: float, z_s: float
169 ) -> np.ndarray:
170     """
171     One-way distance from transmit element(s) to a scatterer at (x_s, z_s).
172
173      $d_{TX}(n) = \sqrt{(x_s - x_n)^2 + z_s^2}$ 
174
175     Parameters
176     -----
177     element_x : array_like
178         TX element x-position(s) [m].
179     x_s, z_s : float
180         Scatterer lateral / axial (depth) position [m].
181
182     Returns
183     -----
184     np.ndarray
185         Distance [m] from each TX element to the scatterer.
186     """
187     element_x = np.asarray(element_x, dtype=float)
188     return np.sqrt((x_s - element_x) ** 2 + z_s ** 2)
189
190
191 def calculate_rx_distance(
192     element_x: np.ndarray, x_s: float, z_s: float
193 ) -> np.ndarray:
194     """
195     One-way distance from a scatterer at (x_s, z_s) to receive element(s).
196
197      $d_{RX}(m) = \sqrt{(x_s - x_m)^2 + z_s^2}$ 
198
199     Geometrically identical to calculate_tx_distance (the array is
200     reciprocal - same elements act as TX and RX here); kept as a
201     separate, explicitly-named function so TX/RX paths stay independently
202     readable/testable and so RX-specific effects (e.g. receive-aperture
203     truncation, element directivity) can be added later without touching
204     the TX path.
205
206     Parameters
207     -----
208     element_x : array_like
209         RX element x-position(s) [m].
210     x_s, z_s : float
211         Scatterer lateral / axial (depth) position [m].
212
213     Returns
214     -----
215     np.ndarray
216         Distance [m] from the scatterer to each RX element.
217     """
218     element_x = np.asarray(element_x, dtype=float)
219     return np.sqrt((x_s - element_x) ** 2 + z_s ** 2)
220
221
222 def calculate_propagation_delay(distance: np.ndarray, c: float = C) -> np.ndarray:
223     """
224     Convert a one-way propagation distance to a propagation time.
225
226      $t = d / C$ 
227
228     Parameters

```

```

229     -----
230     distance : array_like
231         Propagation distance [m].
232     c : float
233         Speed of sound [m/s] (defaults to the global C).
234
235     Returns
236     -----
237     np.ndarray
238         Propagation time [s].
239     """
240     return np.asarray(distance, dtype=float) / c
241
242
243 def calculate_attenuation(
244     distance: np.ndarray, f0: float = F0, alpha0: float = ALPHA_0
245 ) -> np.ndarray:
246     """
247     One-way amplitude attenuation factor for a mechanical pressure wave.
248
249     ALPHA_0 is given in dB / MHz / cm (a standard soft-tissue attenuation
250     convention). To use it we must convert:
251
252         - frequency: Hz -> MHz    (divide by 1e6)
253         - distance:   m   -> cm    (multiply by 100)
254
255     then the attenuation in dB is
256
257         atten_dB = ALPHA_0 * f0_MHz * distance_cm
258
259     and the dB -> linear AMPLITUDE conversion (not power, hence /20 and
260     not /10) is
261
262         atten_linear = 10 ** (-atten_dB / 20)
263
264     Parameters
265     -----
266     distance : array_like
267         One-way propagation distance [m].
268     f0 : float
269         Frequency at which attenuation is evaluated [Hz] (defaults to F0).
270     alpha0 : float
271         Attenuation coefficient [dB/MHz/cm] (defaults to the global ALPHA_0).
272
273     Returns
274     -----
275     np.ndarray
276         Dimensionless linear amplitude attenuation factor in (0, 1].
277     """
278     distance = np.asarray(distance, dtype=float)
279     f0_mhz = f0 / 1.0e6 # Hz -> MHz
280     distance_cm = distance * 100.0 # m -> cm
281     atten_db = alpha0 * f0_mhz * distance_cm
282     atten_linear = 10.0 ** (-atten_db / 20.0)
283     return atten_linear
284
285
286 def forward_propagation(
287     scatterer: Dict[str, float],
288     element_x: np.ndarray,
289     t_axis: np.ndarray,
290     f0: float = F0,
291     c: float = C,
292     alpha0: float = ALPHA_0,
293     n_cycles: int = N_CYCLES,
294     tau_tx: Optional[np.ndarray] = None,
295 ) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
296     """
297     Forward propagation: transducer (all TX elements) -> scatterer.
298
299     The incident pressure field AT THE SCATTERER is the linear
300     superposition of the (delayed, attenuated) pulse contributions
301     arriving from every one of the N_ELEMENTS transmit elements -
302     exactly the "incident mechanical wave" in the module diagram.
303
304         d_TX(n) = sqrt((x_s - x_n)^2 + z_s^2)
305         t_TX(n) = d_TX(n) / C

```

```

306     incident_wave(t) = sum_n A_TX(n) * p(t - tau_TX(n) - t_TX(n))
307
308     ``tau_tx`` is an explicit hook for a future transmit
309     beamforming/focusing delay (t_total = tau_TX(n) + t_TX(n) + t_RX(m)).
310     It defaults to all-zeros ("zero/default transmit delays"), as
311     instructed - no focusing implementation is invented here.
312
313     Parameters
314     -----
315     scatterer : dict
316         Must contain "x" and "z" (scatterer position [m]). "amp" is not
317         used here - amplitude is applied by mock_tissue_interaction().
318     element_x : array_like, shape (n_elements,)
319         TX element x-positions [m] (from create_linear_array()).
320     t_axis : array_like, shape (n_samples,)
321         Shared simulation time axis [s] the incident wave is evaluated on.
322     f0, c, alpha0, n_cycles : float / int
323         Physical parameters, default to the global config values.
324     tau_tx : array_like, optional
325         Per-element transmit delay [s] (beamforming/focusing). Defaults
326         to zeros (no focusing) if not supplied.
327
328     Returns
329     -----
330     incident_wave : np.ndarray, shape (n_samples,)
331         Incident mechanical pressure waveform at the scatterer, sampled
332         on t_axis.
333     d_tx : np.ndarray, shape (n_elements,)
334         One-way TX distance for each element (returned for diagnostics/plots).
335     t_tx : np.ndarray, shape (n_elements,)
336         One-way TX propagation delay for each element (diagnostics/plots).
337     """
338     element_x = np.asarray(element_x, dtype=float)
339     t_axis = np.asarray(t_axis, dtype=float)
340
341     d_tx = calculate_tx_distance(element_x, scatterer["x"], scatterer["z"])
342     t_tx = calculate_propagation_delay(d_tx, c=c)
343     atten_tx = calculate_attenuation(d_tx, f0=f0, alpha0=alpha0)
344
345     if tau_tx is None:
346         tau_tx = np.zeros_like(t_tx)
347     else:
348         tau_tx = np.asarray(tau_tx, dtype=float)
349
350     incident_wave = np.zeros_like(t_axis)
351     for n in range(element_x.size):
352         delay_n = tau_tx[n] + t_tx[n]
353         incident_wave += atten_tx[n] * _pulse_shape(
354             t_axis - delay_n, f0=f0, n_cycles=n_cycles
355         )
356
357     return incident_wave, d_tx, t_tx
358
359
360 def mock_tissue_interaction(
361     incident_wave: np.ndarray, scatterer: Dict[str, float]
362 ) -> np.ndarray:
363     """
364     TEMPORARY synthetic Tissue Interaction interface (Part 2).
365
366     The real Tissue Interaction module (owned by another team) is not
367     ready yet, so this stand-in models the scatterer as simply scaling
368     the incident mechanical wave by its "amp":
369
370         reflected_wave = incident_wave * scatterer["amp"]
371
372     No acoustic impedance, reflection coefficients, or speckle physics
373     are modelled - this is intentionally the simplest possible
374     placeholder. The function's contract (incident wave in, reflected
375     wave out, same length/sampling) is what return_propagation() relies
376     on, so a real Tissue Interaction module can later be swapped in here
377     without changing return_propagation() at all.
378
379     Parameters
380     -----
381     incident_wave : np.ndarray
382         Incident mechanical pressure waveform at the scatterer (output

```

```

383         of forward_propagation()).
384     scatterer : dict
385         Must contain "amp" (dimensionless synthetic reflectivity).
386
387     Returns
388     -----
389     np.ndarray
390         Reflected/scattered mechanical pressure waveform, same shape as
391         incident_wave.
392     """
393     return incident_wave * scatterer["amp"]
394
395
396 def return_propagation(
397     reflected_wave: np.ndarray,
398     t_axis: np.ndarray,
399     scatterer: Dict[str, float],
400     element_x: np.ndarray,
401     f0: float = F0,
402     c: float = C,
403     alpha0: float = ALPHA_0,
404 ) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
405     """
406     Return propagation: scatterer -> every receive element (Part 3).
407
408      $d_{RX}(m) = \sqrt{(x_s - x_m)^2 + z_s^2}$ 
409      $t_{RX}(m) = d_{RX}(m) / C$ 
410      $p_{RX,m}(t) = A_{RX}(m) * p_{scattered}(t - t_{RX}(m))$ 
411
412     ``reflected_wave`` is already a discretely SAMPLED array (it came out
413     of mock_tissue_interaction(), not a closed-form pulse), so shifting it
414     by a continuous delay  $t_{RX}(m)$  is done by linear interpolation onto the
415     shared time axis (np.interp), rather than re-evaluating a formula.
416
417     Parameters
418     -----
419     reflected_wave : np.ndarray, shape (n_samples,)
420         Reflected mechanical pressure waveform at the scatterer, sampled
421         on t_axis (output of mock_tissue_interaction() or, later, a real
422         Tissue Interaction module - this function does not care which).
423     t_axis : np.ndarray, shape (n_samples,)
424         Shared simulation time axis [s].
425     scatterer : dict
426         Must contain "x" and "z" (scatterer position [m]).
427     element_x : array_like, shape (n_elements,)
428         RX element x-positions [m].
429     f0, c, alpha0 : float
430         Physical parameters, default to the global config values.
431
432     Returns
433     -----
434     pressure : np.ndarray, shape (n_elements, n_samples)
435         Returning mechanical acoustic pressure at every receive element.
436     d_rx : np.ndarray, shape (n_elements,)
437         One-way RX distance for each element (diagnostics/plots).
438     t_rx : np.ndarray, shape (n_elements,)
439         One-way RX propagation delay for each element (diagnostics/plots).
440     """
441     element_x = np.asarray(element_x, dtype=float)
442     t_axis = np.asarray(t_axis, dtype=float)
443     reflected_wave = np.asarray(reflected_wave, dtype=float)
444
445     d_rx = calculate_rx_distance(element_x, scatterer["x"], scatterer["z"])
446     t_rx = calculate_propagation_delay(d_rx, c=c)
447     atten_rx = calculate_attenuation(d_rx, f0=f0, alpha0=alpha0)
448
449     n_elements = element_x.size
450     pressure = np.zeros((n_elements, t_axis.size))
451     for m in range(n_elements):
452         # p_scattered(t - t_RX(m)), evaluated on the shared time axis by
453         # interpolating the sampled reflected_wave. Samples that would
454         # fall outside the simulated time window are treated as zero.
455         shifted = np.interp(
456             t_axis - t_rx[m], t_axis, reflected_wave, left=0.0, right=0.0
457         )
458         pressure[m, :] = atten_rx[m] * shifted
459

```

```

460     return pressure, d_rx, t_rx
461
462
463 def simulate_acoustic_propagation(
464     phantom_points: List[Dict[str, float]] = None,
465     n_elements: int = N_ELEMENTS,
466     pitch: float = PITCH,
467     f0: float = F0,
468     fs: float = FS,
469     c: float = C,
470     alpha0: float = ALPHA_0,
471     n_cycles: int = N_CYCLES,
472     time_margin_fraction: float = 0.15,
473 ) -> Dict[str, object]:
474     """
475     Run the complete Phase 1 acoustic propagation pipeline:
476
477     TX elements -> forward_propagation -> mock_tissue_interaction
478                 -> return_propagation -> pressure[64, time]
479
480     for every synthetic scatterer in ``phantom_points``, combining
481     multiple scatterers' echoes by linear superposition at each receive
482     element.
483
484     Parameters
485     -----
486     phantom_points : list of dict, optional
487         Synthetic scatterers, each with "x", "z", "amp". Defaults to the
488         global PHANTOM_POINTS.
489     n_elements, pitch, f0, fs, c, alpha0, n_cycles : physical parameters
490         Default to the global config values.
491     time_margin_fraction : float
492         Extra fraction of the max round-trip time added as head/tail
493         margin on the simulation time axis, so delayed pulses are not
494         clipped.
495
496     Returns
497     -----
498     dict with keys:
499     "t_axis"      : np.ndarray, shape (n_samples,) - shared time axis [s]
500     "pressure"    : np.ndarray, shape (n_elements, n_samples) - FINAL
501                     module output: received mechanical pressure at
502                     every RX element vs time, ALL scatterers summed
503                     (pressure[receive_element, time], per the spec)
504     "received_pressure" : same array as "pressure" (clearer alias)
505     "incident_pressure" : np.ndarray, shape (n_scatterers, n_samples) -
506                     forward_propagation()'s incident wave AT EACH
507                     SCATTERER'S LOCATION vs time, one row per
508                     phantom point (row i <-> phantom_points[i])
509     "reflected_pressure" : np.ndarray, shape (n_scatterers, n_samples) -
510                     mock_tissue_interaction()'s reflected wave AT
511                     EACH SCATTERER'S LOCATION vs time, same row order
512     "element_x"    : np.ndarray, shape (n_elements,) - array geometry
513     "pulse_t"      : np.ndarray - TX pulse time axis (for plotting)
514     "pulse_p"      : np.ndarray - TX pulse waveform (for plotting)
515     "per_scatterer" : list of dict, one per phantom point, each with
516                     "scatterer", "d_tx", "t_tx", "d_rx", "t_rx",
517                     "incident_wave", "reflected_wave" (both at that
518                     scatterer's location vs time), and "pressure"
519                     (that scatterer's own RX contribution, useful
520                     for validating linear superposition)
521     """
522     if phantom_points is None:
523         phantom_points = PHANTOM_POINTS
524
525     element_x = create_linear_array(n_elements=n_elements, pitch=pitch)
526     pulse_t, pulse_p = generate_pulse(fs=fs, f0=f0, n_cycles=n_cycles)
527
528     # Size the shared time axis from the largest total (TX+RX) one-way
529     # delay across all scatterers/elements, so no delayed pulse is clipped.
530     max_total_delay = 0.0
531     for scatterer in phantom_points:
532         d_tx = calculate_tx_distance(element_x, scatterer["x"], scatterer["z"])
533         d_rx = calculate_rx_distance(element_x, scatterer["x"], scatterer["z"])
534         t_tx = calculate_propagation_delay(d_tx, c=c)
535         t_rx = calculate_propagation_delay(d_rx, c=c)
536         max_total_delay = max(max_total_delay, float(np.max(t_tx) + np.max(t_rx)))

```

```

537
538 pulse_half_width = pulse_t[-1] # generate_pulse() is symmetric about 0
539 margin = time_margin_fraction * max_total_delay + pulse_half_width
540 t_start = -pulse_half_width
541 t_end = max_total_delay + margin
542 t_axis = np.arange(t_start, t_end, 1.0 / fs)
543
544 pressure_total = np.zeros((n_elements, t_axis.size))
545 per_scatterer = []
546 incident_list = [] # one incident-wave row per scatterer (per-LOCATION pressure vs time)
547 reflected_list = [] # one reflected-wave row per scatterer
548
549 for scatterer in phantom_points:
550     incident_wave, d_tx, t_tx = forward_propagation(
551         scatterer, element_x, t_axis, f0=f0, c=c, alpha0=alpha0, n_cycles=n_cycles
552     )
553     reflected_wave = mock_tissue_interaction(incident_wave, scatterer)
554     pressure_s, d_rx, t_rx = return_propagation(
555         reflected_wave, t_axis, scatterer, element_x, f0=f0, c=c, alpha0=alpha0
556     )
557
558     pressure_total += pressure_s # linear superposition across scatterers
559     incident_list.append(incident_wave)
560     reflected_list.append(reflected_wave)
561
562     per_scatterer.append(
563         {
564             "scatterer": scatterer,
565             "d_tx": d_tx,
566             "t_tx": t_tx,
567             "d_rx": d_rx,
568             "t_rx": t_rx,
569             "incident_wave": incident_wave, # pressure at THIS scatterer vs time
570             "reflected_wave": reflected_wave, # pressure at THIS scatterer vs time
571             "pressure": pressure_s, # this scatterer's own RX contribution
572         }
573     )
574
575 return {
576     "t_axis": t_axis,
577     # Received mechanical pressure at all 64 RX elements vs time -- the
578     # module's final output, pressure[receive_element, time].
579     "pressure": pressure_total,
580     "received_pressure": pressure_total, # alias, same array, clearer name
581     # Incident/reflected pressure AT EACH SCATTERER LOCATION vs time,
582     # stacked into one array per stage: shape (n_scatterers, n_time).
583     # Row i corresponds to phantom_points[i] / per_scatterer[i].
584     "incident_pressure": np.stack(incident_list, axis=0),
585     "reflected_pressure": np.stack(reflected_list, axis=0),
586     "element_x": element_x,
587     "pulse_t": pulse_t,
588     "pulse_p": pulse_p,
589     "per_scatterer": per_scatterer,
590 }

```

code/nitk_usoundsim/demo_acoustic_propagation.py

```

1  """
2  demo_acoustic_propagation.py
3  =====
4
5  Small end-to-end demo of the Phase-1 acoustic propagation pipeline:
6
7      64 TX elements -> forward_propagation() -> mock_tissue_interaction()
8                      -> return_propagation() -> pressure[64, time]
9
10 run on the three synthetic PHANTOM_POINTS scatterers from config.py.
11 Saves two PNG figures illustrating every stage of the pipeline.
12
13 Run with (from the usound_sim folder, so the package is importable):
14     python -m nitk_usoundsim.demo_acoustic_propagation
15 """
16
17 import os
18
19 import matplotlib

```

```

20 matplotlib.use("Agg")
21 import matplotlib.pyplot as plt
22 import numpy as np
23
24 try:
25     from nitk_usoundsim import config
26     from nitk_usoundsim import acoustic_propagation as ap
27 except ImportError: # allow running directly from inside the package dir
28     import config
29     import acoustic_propagation as ap
30
31 OUT_DIR = os.path.dirname(os.path.abspath(__file__))
32 COLORS = ["tab:blue", "tab:orange", "tab:green"]
33
34
35 def make_figures(out_dir: str = OUT_DIR):
36     result = ap.simulate_acoustic_propagation()
37     x_el = result["element_x"]
38     t_axis = result["t_axis"]
39     pressure = result["pressure"]
40     pulse_t, pulse_p = result["pulse_t"], result["pulse_p"]
41
42     # =====
43     # Figure 1: array geometry, TX pulse, delay-vs-depth, attenuation-vs-depth
44     # =====
45     fig1, axes = plt.subplots(2, 2, figsize=(11, 8))
46
47     # (a) 64-element array geometry + phantom scatterers
48     ax = axes[0, 0]
49     ax.scatter(x_el * 1e3, np.zeros_like(x_el), marker="s", s=18,
50               color="tab:blue", label="TX/RX elements")
51     for pt, c in zip(config.PHANTOM_POINTS, COLORS):
52         ax.scatter(pt["x"] * 1e3, pt["z"] * 1e3, marker="*", s=180, color=c, zorder=3)
53         ax.annotate(f"z={pt['z']*1e3:.0f} mm", (pt["x"] * 1e3, pt["z"] * 1e3),
54                   textcoords="offset points", xytext=(8, 0), fontsize=9)
55     ax.set_xlabel("Lateral x [mm]")
56     ax.set_ylabel("Depth z [mm]")
57     ax.set_title("64-element linear array + phantom scatterers")
58     ax.invert_yaxis()
59     ax.legend(loc="lower right")
60     ax.grid(alpha=0.3)
61
62     # (b) transmitted pulse
63     ax = axes[0, 1]
64     ax.plot(pulse_t * 1e6, pulse_p, color="tab:blue")
65     ax.set_xlabel("Time [\u00b5s]")
66     ax.set_ylabel("Pressure (a.u.)")
67     ax.set_title(f"Transmitted pulse ({config.F0/1e6:.0f} MHz, "
68               f"{ap.N_CYCLES:.0f}-cycle Gaussian envelope)")
69     ax.grid(alpha=0.3)
70
71     # (c) propagation delay vs depth (on-axis, near-central element)
72     x_center = float(x_el[config.N_ELEMENTS // 2])
73     z_sweep = np.linspace(config.Z_MIN, config.Z_MAX, 200)
74     d_sweep = ap.calculate_tx_distance(x_center, 0.0, z_sweep)
75     t_sweep = ap.calculate_propagation_delay(d_sweep, c=config.C)
76
77     ax = axes[1, 0]
78     ax.plot(z_sweep * 1e3, t_sweep * 1e6, color="tab:green")
79     for pt, c in zip(config.PHANTOM_POINTS, COLORS):
80         ax.axvline(pt["z"] * 1e3, color=c, ls="--", alpha=0.6)
81     ax.set_xlabel("Depth z [mm]")
82     ax.set_ylabel("One-way delay [\u00b5s]")
83     ax.set_title("Propagation delay vs depth (on-axis)")
84     ax.grid(alpha=0.3)
85
86     # (d) attenuation vs depth
87     a_sweep = ap.calculate_attenuation(d_sweep, f0=config.F0, alpha0=config.ALPHA_0)
88     a_sweep_db = -20.0 * np.log10(a_sweep)
89
90     ax = axes[1, 1]
91     ax.plot(z_sweep * 1e3, a_sweep_db, color="tab:purple")
92     for pt, c in zip(config.PHANTOM_POINTS, COLORS):
93         ax.axvline(pt["z"] * 1e3, color=c, ls="--", alpha=0.6)
94     ax.set_xlabel("Depth z [mm]")
95     ax.set_ylabel("One-way attenuation [dB]")
96     ax.set_title(f"Attenuation vs depth (at F0 = {config.F0/1e6:.0f} MHz)")

```

```

97     ax.grid(alpha=0.3)
98
99     fig1.tight_layout()
100    fig1_path = os.path.join(out_dir, "demo_geometry_pulse_delay_attenuation.png")
101    fig1.savefig(fig1_path, dpi=140)
102    plt.close(fig1)
103
104    # =====
105    # Figure 2: pipeline signals (incident / reflected / one RX / RX image)
106    # =====
107    fig2, axes2 = plt.subplots(2, 2, figsize=(11, 8))
108
109    # (a) incident wave AT each scatterer (forward_propagation output)
110    ax = axes2[0, 0]
111    incidents = {}
112    for entry, c in zip(result["per_scatterer"], COLORS):
113        scatterer = entry["scatterer"]
114        incident, _, _ = ap.forward_propagation(scatterer, x_el, t_axis)
115        incidents[id(scatterer)] = incident
116        ax.plot(t_axis * 1e6, incident, color=c, label=f"z={scatterer['z']*1e3:.0f} mm")
117    ax.set_xlabel("Time [\u00b5s]")
118    ax.set_ylabel("Incident pressure (a.u.)")
119    ax.set_title("Forward propagation: incident wave at each scatterer")
120    ax.legend()
121    ax.grid(alpha=0.3)
122
123    # (b) synthetic reflected/scattered waves (mock_tissue_interaction output)
124    ax = axes2[0, 1]
125    for entry, c in zip(result["per_scatterer"], COLORS):
126        scatterer = entry["scatterer"]
127        reflected = ap.mock_tissue_interaction(incidents[id(scatterer)], scatterer)
128        ax.plot(t_axis * 1e6, reflected, color=c,
129               label=f"z={scatterer['z']*1e3:.0f} mm, amp={scatterer['amp']:.1f}")
130    ax.set_xlabel("Time [\u00b5s]")
131    ax.set_ylabel("Reflected pressure (a.u.)")
132    ax.set_title("Synthetic (mock) reflected/scattered waves")
133    ax.legend()
134    ax.grid(alpha=0.3)
135
136    # (c) one receive-element pressure waveform (central element)
137    ax = axes2[1, 0]
138    m_center = config.N_ELEMENTS // 2
139    ax.plot(t_axis * 1e6, pressure[m_center, :], color="black")
140    for pt, c in zip(config.PHANTOM_POINTS, COLORS):
141        expected_rt = 2 * pt["z"] / config.C
142        ax.axvline(expected_rt * 1e6, color=c, ls=":", alpha=0.6)
143    ax.set_xlabel("Time [\u00b5s]")
144    ax.set_ylabel("Pressure (a.u.)")
145    ax.set_title(f"Return propagation: RX element #{m_center} waveform")
146    ax.text(0.02, 0.02, "dotted = expected same-path\nround-trip time per target",
147           transform=ax.transAxes, fontsize=8, va="bottom", ha="left", alpha=0.8)
148    ax.grid(alpha=0.3)
149
150    # (d) receive-element x time image of the full returning pressure field
151    ax = axes2[1, 1]
152    extent = [t_axis[0] * 1e6, t_axis[-1] * 1e6, config.N_ELEMENTS - 1, 0]
153    vmax = np.max(np.abs(pressure)) or 1.0
154    im = ax.imshow(pressure, aspect="auto", extent=extent, cmap="seismic",
155                  vmin=-vmax, vmax=vmax)
156    ax.set_xlabel("Time [\u00b5s]")
157    ax.set_ylabel("RX element index")
158    ax.set_title("Returning mechanical pressure field (all 64 RX elements)")
159    fig2.colorbar(im, ax=ax, label="Pressure (a.u.)")
160
161    fig2.tight_layout()
162    fig2_path = os.path.join(out_dir, "demo_pipeline_signals.png")
163    fig2.savefig(fig2_path, dpi=140)
164    plt.close(fig2)
165
166    return fig1_path, fig2_path, result
167
168
169 def save_part_outputs(result, out_dir: str = OUT_DIR):
170     """
171     Save this module's two deliverables into SEPARATE, self-describing
172     .npz files -- one per project part -- so each can be shown/inspected
173     independently:

```

```

174
175     PART 1  forward_propagation()  TX array -> scatterer
176           -> part1_forward_propagation_outputs.npz
177
178     PART 2  return_propagation()    scatterer -> RX array (FINAL output)
179           -> part2_return_propagation_outputs.npz
180
181     Every numeric array is paired with a "*_units" and/or "*_row_labels"
182     entry, and each file has a top-level "description" string, so the
183     contents are understandable on their own when printed -- no need to
184     have this source file open alongside them.
185     ""
186     scatterer_labels = np.array([
187         f"scatterer_{i}: x={e['scatterer']['x']*1e3:.1f}mm, "
188         f"z={e['scatterer']['z']*1e3:.1f}mm, amp={e['scatterer']['amp']:.2f}"
189         for i, e in enumerate(result["per_scatterer"])
190     ])
191     scatterer_xyz = np.array(
192         [[e["scatterer"]["x"], e["scatterer"]["z"], e["scatterer"]["amp"]]
193          for e in result["per_scatterer"]]
194     )
195     rx_labels = np.array(
196         [f"rx_element_{m:02d}" for m in range(result["received_pressure"].shape[0])]
197     )
198
199     # ----- PART 1: forward propagation (TX array -> scatterer) ----
200     part1_path = os.path.join(out_dir, "part1_forward_propagation_outputs.npz")
201     np.savez(
202         part1_path,
203         description=np.array(
204             "PART 1 - forward_propagation(): incident mechanical pressure wave "
205             "arriving AT EACH SCATTERER from the 64-element TX array, vs time. "
206             "Row i of incident_pressure corresponds to scatterer_xyz[i] / "
207             "incident_pressure_row_labels[i]. Also included: the TX pulse "
208             "(pulse_t, pulse_p) and TX element geometry (element_x) used to "
209             "compute it."
210         ),
211         t_axis=result["t_axis"],
212         t_axis_units=np.array("seconds"),
213         incident_pressure=result["incident_pressure"],
214         incident_pressure_shape_meaning=np.array("(n_scatterers, n_time_samples)"),
215         incident_pressure_units=np.array("arbitrary pressure units (a.u.)"),
216         incident_pressure_row_labels=scatterer_labels,
217         scatterer_xyz=scatterer_xyz,
218         scatterer_xyz_columns=np.array(["x [m]", "z [m]", "amp"]),
219         element_x=result["element_x"],
220         element_x_units=np.array("meters (TX element lateral position)"),
221         pulse_t=result["pulse_t"],
222         pulse_p=result["pulse_p"],
223     )
224
225     # ----- PART 2: return propagation (scatterer -> RX array) -----
226     part2_path = os.path.join(out_dir, "part2_return_propagation_outputs.npz")
227     np.savez(
228         part2_path,
229         description=np.array(
230             "PART 2 - return_propagation(): FINAL module output. Mechanical "
231             "pressure received at each of the 64 RX elements vs time "
232             "(received_pressure), all scatterers combined by linear "
233             "superposition. reflected_pressure_input is the reflected/"
234             "scattered wave this stage propagates to the RX array -- it is "
235             "INCLUDED HERE ONLY FOR CONTEXT (currently produced by the "
236             "temporary mock tissue-interaction stand-in; return_propagation() "
237             "itself does not care how it was produced, so the real tissue-"
238             "interaction module's output can replace it later unchanged)."
239         ),
240         t_axis=result["t_axis"],
241         t_axis_units=np.array("seconds"),
242         received_pressure=result["received_pressure"],
243         received_pressure_shape_meaning=np.array("(64 rx_elements, n_time_samples)"),
244         received_pressure_units=np.array("arbitrary pressure units (a.u.)"),
245         received_pressure_row_labels=rx_labels,
246         reflected_pressure_input=result["reflected_pressure"],
247         reflected_pressure_input_row_labels=scatterer_labels,
248         scatterer_xyz=scatterer_xyz,
249         scatterer_xyz_columns=np.array(["x [m]", "z [m]", "amp"]),
250         element_x=result["element_x"],

```

```

251     element_x_units=np.array("meters (RX element lateral position)"),
252 )
253
254     return part1_path, part2_path
255
256
257 if __name__ == "__main__":
258     f1, f2, result = make_figures()
259     part1_path, part2_path = save_part_outputs(result)
260
261     print("Saved figure:", f1)
262     print("Saved figure:", f2)
263     print("Saved PART 1 (forward propagation) data:", part1_path)
264     print("Saved PART 2 (return propagation) data: ", part2_path)
265     print()
266     print("PART 1 - incident_pressure :", result["incident_pressure"].shape,
267           " <- pressure AT each scatterer, per phantom point")
268     print("PART 2 - received_pressure :", result["received_pressure"].shape,
269           " <- FINAL output, pressure[r_x_element, time]")
270     print()
271     print("To load and inspect later, e.g.:")
272     print(" import numpy as np")
273     print(" d1 = np.load('part1_forward_propagation_outputs.npz')")
274     print(" print(d1['description'])")
275     print(" print(d1.files)                # lists every labelled array")
276     print(" d1['incident_pressure']         # (n_scatterers, n_time)")
277     print(" d1['incident_pressure_row_labels'] # which row is which scatterer")
278     print()
279     print(" d2 = np.load('part2_return_propagation_outputs.npz')")
280     print(" print(d2['description'])")
281     print(" d2['received_pressure']           # (64, n_time)")

```

code/nitk_usoundsim/test_acoustic_propagation.py

```

1  """
2  test_acoustic_propagation.py
3  =====
4
5  Basic unit tests for the Phase-1 Acoustic Propagation module
6  (nitk_usoundsim.acoustic_propagation), covering:
7
8  - element positions
9  - wavelength and pitch
10 - TX distance calculation
11 - RX distance calculation
12 - TX propagation delay
13 - RX propagation delay
14 - attenuation calculation
15 - output dimensions
16 - expected round-trip delay for the 20 mm phantom target
17 - deeper targets -> larger delay / more attenuation (monotonicity)
18 - off-axis targets -> different RX delays across elements
19 - multiple-scatterer linear superposition
20
21 Run with: python -m unittest nitk_usoundsim.test_acoustic_propagation -v
22 (from the usound_sim folder, so the `nitk_usoundsim` package is importable).
23 """
24
25 import unittest
26
27 import numpy as np
28
29 from nitk_usoundsim import config
30 from nitk_usoundsim import acoustic_propagation as ap
31
32
33 class TestArrayGeometry(unittest.TestCase):
34     def test_element_count(self):
35         x = ap.create_linear_array()
36         self.assertEqual(x.size, config.N_ELEMENTS)
37
38     def test_element_spacing_equals_pitch(self):
39         x = ap.create_linear_array()
40         spacing = np.diff(x)
41         np.testing.assert_allclose(spacing, config.PITCH, rtol=1e-12)
42

```

```

43     def test_array_centred_on_zero(self):
44         x = ap.create_linear_array()
45         self.assertAlmostEqual(float(np.mean(x)), 0.0, places=12)
46
47     def test_wavelength_and_pitch_relationship(self):
48         # LAMBDA = C / F0 ; PITCH = LAMBDA / 2 (both come from config.py)
49         expected_lambda = config.C / config.F0
50         self.assertAlmostEqual(config.LAMBDA, expected_lambda, places=9)
51         self.assertAlmostEqual(config.PITCH, expected_lambda / 2.0, places=9)
52
53
54 class TestDistancesAndDelays(unittest.TestCase):
55     def setUp(self):
56         self.x_elements = ap.create_linear_array()
57
58     def test_tx_distance_matches_manual_formula(self):
59         x_s, z_s = 0.003, 0.020
60         d = ap.calculate_tx_distance(self.x_elements, x_s, z_s)
61         expected = np.sqrt((x_s - self.x_elements) ** 2 + z_s ** 2)
62         np.testing.assert_allclose(d, expected, rtol=1e-12)
63
64     def test_rx_distance_matches_manual_formula(self):
65         x_s, z_s = -0.004, 0.030
66         d = ap.calculate_rx_distance(self.x_elements, x_s, z_s)
67         expected = np.sqrt((x_s - self.x_elements) ** 2 + z_s ** 2)
68         np.testing.assert_allclose(d, expected, rtol=1e-12)
69
70     def test_tx_distance_on_axis_center_equals_depth(self):
71         # For a scatterer directly on axis (x_s=0), the two central
72         # elements should be almost exactly `z_s` away.
73         d = ap.calculate_tx_distance(self.x_elements, 0.0, 0.020)
74         self.assertAlmostEqual(d.min(), 0.020, delta=1e-4)
75
76     def test_tx_propagation_delay(self):
77         d = np.array([0.010, 0.020, 0.030])
78         t = ap.calculate_propagation_delay(d, c=config.C)
79         np.testing.assert_allclose(t, d / config.C, rtol=1e-12)
80
81     def test_rx_propagation_delay(self):
82         d = np.array([0.015, 0.025])
83         t = ap.calculate_propagation_delay(d, c=config.C)
84         np.testing.assert_allclose(t, d / config.C, rtol=1e-12)
85
86     def test_deeper_targets_have_larger_tx_delay(self):
87         d_shallow = ap.calculate_tx_distance(self.x_elements, 0.0, 0.010)
88         d_deep = ap.calculate_tx_distance(self.x_elements, 0.0, 0.030)
89         t_shallow = ap.calculate_propagation_delay(d_shallow, c=config.C)
90         t_deep = ap.calculate_propagation_delay(d_deep, c=config.C)
91         self.assertTrue(np.all(t_deep > t_shallow))
92
93     def test_offaxis_target_gives_varying_rx_delays_across_elements(self):
94         d = ap.calculate_rx_distance(self.x_elements, 0.005, 0.020)
95         t = ap.calculate_propagation_delay(d, c=config.C)
96         # An off-axis target must not arrive at every element at the same time.
97         self.assertGreater(t.max() - t.min(), 0.0)
98
99
100 class TestAttenuation(unittest.TestCase):
101     def test_attenuation_matches_manual_db_formula(self):
102         distance_m = 0.020
103         atten = ap.calculate_attenuation(distance_m, f0=config.F0, alpha0=config.ALPHA_0)
104         f_mhz = config.F0 / 1.0e6
105         d_cm = distance_m * 100.0
106         expected_db = config.ALPHA_0 * f_mhz * d_cm
107         expected_linear = 10.0 ** (-expected_db / 20.0)
108         self.assertAlmostEqual(atten, expected_linear, places=12)
109
110     def test_attenuation_at_20mm_is_5dB(self):
111         # ALPHA_0=0.5 dB/MHz/cm * F0=5MHz * 2cm = 5 dB one-way.
112         atten = ap.calculate_attenuation(0.020, f0=config.F0, alpha0=config.ALPHA_0)
113         atten_db = -20.0 * np.log10(atten)
114         self.assertAlmostEqual(atten_db, 5.0, places=6)
115
116     def test_deeper_targets_experience_more_attenuation(self):
117         atten_shallow = ap.calculate_attenuation(0.010)
118         atten_deep = ap.calculate_attenuation(0.030)
119         # Less remaining amplitude (smaller linear factor) means MORE attenuation.

```

```

120         self.assertLess(atten_deep, atten_shallow)
121
122     def test_attenuation_is_bounded_between_0_and_1(self):
123         atten = ap.calculate_attenuation(np.array([0.0, 0.01, 0.02, 0.05]))
124         self.assertTrue(np.all(atten <= 1.0))
125         self.assertTrue(np.all(atten > 0.0))
126
127 class TestPulse(unittest.TestCase):
128     def test_pulse_sampled_at_fs(self):
129         t, p = ap.generate_pulse()
130         dt = np.diff(t)
131         np.testing.assert_allclose(dt, 1.0 / config.FS, rtol=1e-9)
132         self.assertEqual(t.shape, p.shape)
133
134     def test_pulse_centered_near_zero(self):
135         t, p = ap.generate_pulse()
136         self.assertAlmostEqual(t[np.argmax(np.abs(p))], 0.0, delta=2.0 / config.FS)
137
138 class TestFullPipeline(unittest.TestCase):
139     @classmethod
140     def setUpClass(cls):
141         cls.result = ap.simulate_acoustic_propagation()
142
143     def test_output_dimensions(self):
144         pressure = self.result["pressure"]
145         self.assertEqual(pressure.shape[0], config.N_ELEMENTS)
146         self.assertEqual(pressure.shape[1], self.result["t_axis"].size)
147         self.assertGreater(pressure.shape[1], 0)
148
149     def test_output_is_real_valued_mechanical_pressure(self):
150         pressure = self.result["pressure"]
151         self.assertTrue(np.isrealobj(pressure))
152         self.assertTrue(np.all(np.isfinite(pressure)))
153
154     def test_round_trip_delay_for_20mm_phantom_point(self):
155         # Nominal on-axis, same-path round trip: 2 * 20e-3 / 1540 ~ 25.97 us
156         entry_20mm = next(
157             e for e in self.result["per_scatterer"] if abs(e["scatterer"]["z"] - 0.020) < 1e-9
158         )
159         round_trip_min = entry_20mm["t_tx"].min() + entry_20mm["t_rx"].min()
160         expected = 2.0 * 0.020 / config.C
161         self.assertAlmostEqual(round_trip_min, expected, delta=0.05e-6) # 50 ns tolerance
162
163     def test_deeper_phantom_points_have_larger_min_delay(self):
164         entries = sorted(self.result["per_scatterer"], key=lambda e: e["scatterer"]["z"])
165         min_delays = [e["t_tx"].min() for e in entries]
166         self.assertTrue(np.all(np.diff(min_delays) > 0))
167
168     def test_multiple_scatterer_linear_superposition(self):
169         total = self.result["pressure"]
170         manual_sum = np.zeros_like(total)
171         for entry in self.result["per_scatterer"]:
172             manual_sum += entry["pressure"]
173         np.testing.assert_allclose(total, manual_sum, rtol=1e-12, atol=1e-15)
174
175 if __name__ == "__main__":
176     unittest.main(verbosity=2)

```

code/nitk_usoundsim/check_arr.py

```

1 import numpy as np
2 #data = np.load("nitk_usoundsim/acoustic_propagation_outputs.npz")
3 #print(data["received_pressure"])
4
5 '''data = np.load("nitk_usoundsim/part1_forward_propagation_outputs.npz")
6 print(data["description"])
7 print(data.files)'''
8
9
10 data = np.load("nitk_usoundsim/part2_return_propagation_outputs.npz")
11 print(data["description"])
12 print(data["reflected_pressure_input"])
13 print(data["reflected_pressure_input"].shape)

```

```
14 print(data["reflected_pressure_input_row_labels"])
```

code/nitk_usoundsim/requirements.txt

```
1 numpy
2 matplotlib
```

run_team3.py (runner added for this demo)

```
1 """
2 run_team3.py - Team 3: Acoustic Propagation (NITK-UsoundSim)
3 =====
4
5 ONE command runs the whole Team 3 demo:
6
7     python3 run_team3.py      (macOS / Linux)
8     python  run_team3.py      (Windows)
9
10 What it does (the team's code in code/nitk_usoundsim/ is NOT changed;
11 this script only imports it and calls its functions):
12
13 STEP 1 Run the team's 22 unit tests (test_acoustic_propagation.py).
14 STEP 2 Run the team's own demo (64 elements, 5 MHz, 3 scatterers)
15        and save its two figures + two .npz data files into output/.
16 STEP 3 Second run with the project's final probe (Philips L12-4 model:
17        128 elements, 0.30 mm pitch, 8 MHz) and one point scatterer at
18        (0, 20) mm, done only by passing arguments to the team's functions.
19 STEP 4 Attenuation vs depth at 4, 8 and 12 MHz (team's calculate_attenuation).
20
21 Everything is saved in output/ (plots, data, console log, console screenshot).
22 """
23
24 import io
25 import json
26 import os
27 import subprocess
28 import sys
29 import time
30
31 sys.dont_write_bytecode = True          # keep code/ clean (no __pycache__)
32
33 HERE = os.path.dirname(os.path.abspath(__file__))
34 CODE_DIR = os.path.join(HERE, "code")
35 INPUT_FILE = os.path.join(HERE, "input", "team3_run_parameters.json")
36 OUT_DIR = os.path.join(HERE, "output")
37 sys.path.insert(0, CODE_DIR)           # makes the team's package importable
38
39 import matplotlib                        # noqa: E402
40 matplotlib.use("Agg")
41 import matplotlib.pyplot as plt          # noqa: E402
42 import numpy as np                      # noqa: E402
43
44 from nitk_usoundsim import config        # noqa: E402 (team code)
45 from nitk_usoundsim import acoustic_propagation as ap    # noqa: E402 (team code)
46 from nitk_usoundsim import demo_acoustic_propagation as demo # noqa: E402 (team code)
47
48
49 # -----
50 # Small helpers (printing to screen AND to a log file)
51 # -----
52 class Tee:
53     """Send printed text to the real console and keep a copy for the log."""
54
55     def __init__(self, stream):
56         self.stream = stream
57         self.buffer = io.StringIO()
58
59     def write(self, text):
60         self.stream.write(text)
61         self.buffer.write(text)
62
63     def flush(self):
64         self.stream.flush()
65
```

```

66
67 def banner(title):
68     print()
69     print("=" * 74)
70     print(title)
71     print("=" * 74)
72
73
74 def rel(path):
75     return os.path.relpath(path, HERE)
76
77
78 def console_to_png(text, out_png, title="Terminal", width_chars=100, max_lines=150):
79     """Draw the captured console text as a terminal-style screenshot PNG."""
80     lines = []
81     for line in text.rstrip("\n").expandtabs(4).splitlines() or [""]:
82         while len(line) > width_chars:
83             lines.append(line[:width_chars])
84             line = " -> " + line[width_chars:]
85         lines.append(line)
86     if len(lines) > max_lines:
87         lines = lines[:max_lines - 1] + [f"... ({len(lines) - max_lines + 1} more lines, see conso
88 le_log.txt)"]
89     h = 0.62 + 0.165 * len(lines)
90     fig = plt.figure(figsize=(0.083 * width_chars + 0.4, h))
91     fig.patch.set_facecolor("#1e1e1e")
92     fig.text(0.012, 1 - 0.28 / h, "●●● " + title, color="#9a9a9a",
93             fontsize=9, family="DejaVu Sans Mono", va="center")
94     for i, ln in enumerate(lines):
95         fig.text(0.012, 1 - (0.55 + 0.165 * i) / h, ln, color="#e6e6e6",
96                 fontsize=8.6, family="DejaVu Sans Mono", va="top")
97     fig.savefig(out_png, dpi=160, facecolor=fig.get_facecolor())
98     plt.close(fig)
99     return out_png
100
101
102 # -----
103 # STEP 1: the team's unit tests
104 # -----
105 def step1_unit_tests():
106     banner("STEP 1: Team 3 unit tests (test_acoustic_propagation.py)")
107     print("Command used: python -m unittest nitk_usoundsim.test_acoustic_propagation -v")
108     print("(run inside the code/ folder)\n")
109     env = dict(os.environ, PYTHONDONTWRITEBYTECODE="1")
110     proc = subprocess.run(
111         [sys.executable, "-m", "unittest", "nitk_usoundsim.test_acoustic_propagation", "-v"],
112         cwd=CODE_DIR, env=env, capture_output=True, text=True)
113     text = (proc.stdout + proc.stderr).strip()
114     with open(os.path.join(OUT_DIR, "unit_tests_full_output.txt"), "w") as fh:
115         fh.write(text + "\n")
116     # Print a short line per test: "ok Class.test_name" (full text is saved)
117     for line in text.splitlines():
118         if " ... " in line and "(" in line:
119             name, _, verdict = line.partition(" ... ")
120             full = name[name.find("(") + 1:name.rfind(")")]
121             short = ".".join(full.split(".")[:-2])
122             print(f" {verdict.strip():5s} {short}")
123         elif line.startswith("Ran ") or line.strip() == "OK" or line.startswith("FAILED"):
124             print(line)
125     print(" (full unittest text saved in output/unit_tests_full_output.txt)")
126     n_ran = 0
127     for line in text.splitlines():
128         if line.startswith("Ran ") and " test" in line:
129             n_ran = int(line.split()[1])
130     n_ok = sum(1 for line in text.splitlines() if line.rstrip().endswith("... ok"))
131     status = "ALL PASSED" if proc.returncode == 0 else "SOME TESTS FAILED"
132     print(f"\nRESULT: {n_ok} of {n_ran} tests passed -> {status}")
133     return n_ok, n_ran
134
135
136 # -----
137 # STEP 2: the team's own demo (64 elements, 5 MHz, 3 scatterers)
138 # -----
139 def step2_team_demo(params):
140     banner("STEP 2: Team demo - 64 elements, 5 MHz, lambda/2 pitch (config.py)")
141     p1 = params["run1_team_design"]
142     same = (config.N_ELEMENTS == p1["n_elements"] and config.F0 == p1["f0_hz"])

```

```

143         and config.FS == p1["fs_hz"] and config.C == p1["c_m_per_s"]
144         and config.ALPHA_0 == p1["alpha0_db_per_mhz_per_cm"]
145         and config.PHANTOM_POINTS == p1["scatterers"])
146     print(f"Input check: values in input JSON match config.py -> {same}")
147     print(f"Elements      : {config.N_ELEMENTS}")
148     print(f"Centre freq   : {config.F0 / 1e6:.1f} MHz")
149     print(f"Sampling freq : {config.FS / 1e6:.1f} MHz")
150     print(f"Speed of sound: {config.C:.0f} m/s")
151     print(f"Wavelength   : {config.LAMBDA * 1e3:.4f} mm")
152     print(f"Pitch (lam/2) : {config.PITCH * 1e3:.4f} mm")
153     print(f"Attenuation    : {config.ALPHA_0} dB/MHz/cm")
154     print(f"Scatterers     : " + ", ".join(
155         f"({p['x'] * 1e3:.0f}, {p['z'] * 1e3:.0f}) mm" for p in config.PHANTOM_POINTS))
156
157     # The team's demo normally saves next to its own source file. Here we pass
158     # out_dir=output/ to its functions so the original folder stays untouched.
159     f1, f2, result = demo.make_figures(out_dir=OUT_DIR)
160     part1, part2 = demo.save_part_outputs(result, out_dir=OUT_DIR)
161     print()
162     print("Saved figure:", rel(f1))
163     print("Saved figure:", rel(f2))
164     print("Saved PART 1 (forward propagation) data:", rel(part1))
165     print("Saved PART 2 (return propagation) data: ", rel(part2))
166
167     t = result["t_axis"]
168     print()
169     print(f"Time axis      : {t.size} samples, {t[0] * 1e6:.3f} to {t[-1] * 1e6:.3f} us")
170     print(f"PART 1 incident_pressure shape : {result['incident_pressure'].shape}")
171         " (scatterers, time)")
172     print(f"PART 2 received_pressure shape : {result['received_pressure'].shape}")
173         " (RX elements, time)")
174     print("Per scatterer (on-axis, round trip = min t_TX + min t_RX):")
175     for e in result["per_scatterer"]:
176         s = e["scatterer"]
177         rt = (e["t_tx"].min() + e["t_rx"].min()) * 1e6
178         a = ap.calculate_attenuation(e["d_tx"].min()) ** 2
179         print(f" z = {s['z'] * 1e3:2.0f} mm: one-way t = {e['t_tx'].min() * 1e6:6.3f}-"
180             f"{e['t_tx'].max() * 1e6:6.3f} us | round trip = {rt:6.3f} us "
181             f"{2*zc = {2 * s['z'] / config.C * 1e6:6.3f}} | 2-way amp = {a:.3f}")
182     return result
183
184
185 # -----
186 # STEP 3: project's final probe (Philips L12-4 model), by arguments only
187 # -----
188 def step3_final_probe(params):
189     p = params["run2_final_probe"]
190     banner("STEP 3: Final probe - Philips L12-4 model (128 el, 0.30 mm, 8 MHz)")
191     print("Done ONLY by passing arguments to the team's functions (code unchanged).")
192     n_el, pitch, f0, fs = p["n_elements"], p["pitch_m"], p["f0_hz"], p["fs_hz"]
193     scat = dict(p["scatterer"])
194
195     element_x = ap.create_linear_array(n_el, pitch)
196     pulse_t, pulse_p = ap.generate_pulse(fs=fs, f0=f0)
197     half_width = pulse_t[-1]
198
199     # Build the shared time axis the same way the team's simulator does:
200     # start at minus the pulse half-width, end after the longest round trip.
201     t_tx_all = ap.calculate_propagation_delay(
202         ap.calculate_tx_distance(element_x, scat["x"], scat["z"]))
203     t_rx_all = ap.calculate_propagation_delay(
204         ap.calculate_rx_distance(element_x, scat["x"], scat["z"]))
205     max_total = float(t_tx_all.max() + t_rx_all.max())
206     t_end = max_total + p["time_margin_fraction"] * max_total + half_width
207     t_axis = np.arange(-half_width, t_end, 1.0 / fs)
208
209     incident, d_tx, t_tx = ap.forward_propagation(scat, element_x, t_axis, f0=f0)
210     reflected = ap.mock_tissue_interaction(incident, scat)
211     pressure, d_rx, t_rx = ap.return_propagation(reflected, t_axis, scat, element_x, f0=f0)
212
213     print(f"Elements      : {n_el} pitch = {pitch * 1e3:.2f} mm "
214         f"aperture = {(element_x[-1] - element_x[0]) * 1e3:.2f} mm")
215     print(f"Element x range : {element_x[0] * 1e3:.2f} to {element_x[-1] * 1e3:.2f} mm")
216     print(f"Pulse          : {f0 / 1e6:.0f} MHz, {ap.N_CYCLES} cycles, sigma = "
217         f"{ap.N_CYCLES / (2 * f0) * 1e6:.4f} us, half-width = {half_width * 1e6:.4f} us, "
218         f"{pulse_t.size} samples")
219     print(f"Scatterer       : ({scat['x'] * 1e3:.0f}, {scat['z'] * 1e3:.0f}) mm, amp = {scat['am

```

```

220     p']})")
221     print(f"   Time axis       : {t_axis.size} samples at {fs / 1e6:.0f} MHz, "
222           f"{t_axis[0] * 1e6:.3f} to {t_axis[-1] * 1e6:.3f} us")
223     print(f"   One-way distance: {d_tx.min() * 1e3:.3f} to {d_tx.max() * 1e3:.3f} mm")
224     print(f"   One-way time   : {t_tx.min() * 1e6:.2f} to {t_tx.max() * 1e6:.2f} us "
225           f"(centre to edge elements)")
226     print(f"   Round trip time : {(t_tx.min() + t_rx.min()) * 1e6:.2f} us (centre element) to "
227           f"{(t_tx.max() + t_rx.max()) * 1e6:.2f} us (edge element)")
228     a = ap.calculate_attenuation(d_tx, f0=f0)
229     print(f"   One-way amplitude factor at 8 MHz: {a.max():.4f} (centre) to {a.min():.4f} (edge)"
230           f" = {-20 * np.log10(a.max()):.2f} to {-20 * np.log10(a.min()):.2f} dB")
231     i_pk = int(np.argmax(np.abs(incident)))
232     print(f"   Incident wave peak |p| = {abs(incident[i_pk]):.3f} a.u. at t = {t_axis[i_pk] * 1e6:.
233           2f} us"
234           f" (sum of {n_el} attenuated pulses)")
235     print(f"   Received pressure array shape: {pressure.shape} (RX elements, time)")
236     m_c = n_el // 2
237     j_pk = int(np.argmax(np.abs(pressure[m_c])))
238     print(f"   Element #{m_c}: peak |p| = {abs(pressure[m_c, j_pk]):.3f} a.u. at t = {t_axis[j_pk]
239           * 1e6:.2f} us")
240     print(f"   Element #0 : peak |p| = {np.abs(pressure[0]).max():.3f} a.u. at t = "
241           f"{t_axis[np.argmax(np.abs(pressure[0]))] * 1e6:.2f} us")
242
243     # ----- figure -----
244     fig, axs = plt.subplots(2, 2, figsize=(12, 8.5))
245     ax = axs[0, 0]
246     ax.plot(pulse_t * 1e6, pulse_p, color="tab:blue")
247     ax.set_title(f"(a) Transmitted pulse: {f0 / 1e6:.0f} MHz, {ap.N_CYCLES}-cycle Gaussian")
248     ax.set_xlabel("Time [μs]"); ax.set_ylabel("Pressure (a.u.)"); ax.grid(alpha=0.3)
249
250     ax = axs[0, 1]
251     ax.plot(t_axis * 1e6, incident, color="tab:orange", lw=0.9)
252     ax.axvspan(t_tx.min() * 1e6, t_tx.max() * 1e6, color="grey", alpha=0.15,
253               label="range of one-way travel times")
254     ax.set_xlim((t_tx.min() - 1.5e-6) * 1e6, (t_tx.max() + 1.5e-6) * 1e6)
255     ax.set_title("(b) Incident wave at the scatterer (0, 20) mm")
256     ax.set_xlabel("Time [μs]"); ax.set_ylabel("Pressure (a.u.)")
257     ax.legend(loc="upper right", fontsize=8); ax.grid(alpha=0.3)
258
259     ax = axs[1, 0]
260     idx = np.arange(n_el)
261     ax.plot(idx, t_tx * 1e6, color="tab:green", label="t_TX (element -> scatterer)")
262     ax.plot(idx, t_rx * 1e6, "k:", label="t_RX (scatterer -> element)")
263     ax.set_title("(c) One-way travel time vs element")
264     ax.set_xlabel("Element index"); ax.set_ylabel("Time [μs]")
265     ax.legend(fontsize=8); ax.grid(alpha=0.3)
266
267     ax = axs[1, 1]
268     vmax = np.max(np.abs(pressure)) or 1.0
269     im = ax.imshow(pressure, aspect="auto", cmap="seismic", vmin=-vmax, vmax=vmax,
270                   extent=[t_axis[0] * 1e6, t_axis[-1] * 1e6, n_el - 1, 0])
271     ax.set_xlim(24, 38)
272     ax.set_title(f"(d) Echo received on all {n_el} elements")
273     ax.set_xlabel("Time [μs]"); ax.set_ylabel("RX element index")
274     fig.colorbar(im, ax=ax, label="Pressure (a.u.)")
275     fig.suptitle("Final probe run: Philips L12-4 model, point scatterer at (0, 20) mm", fontweight
276                 ="bold")
277     fig.tight_layout()
278     fig_path = os.path.join(OUT_DIR, "L12-4_final_probe_propagation.png")
279     fig.savefig(fig_path, dpi=140)
280     plt.close(fig)
281
282     npz_path = os.path.join(OUT_DIR, "L12-4_final_probe_outputs.npz")
283     np.savez(npz_path, t_axis=t_axis, element_x=element_x, pulse_t=pulse_t, pulse_p=pulse_p,
284             incident_wave=incident, reflected_wave=reflected, received_pressure=pressure,
285             d_tx=d_tx, t_tx=t_tx, d_rx=d_rx, t_rx=t_rx,
286             description=np.array("Final probe run (128 el, 0.30 mm pitch, 8 MHz), scatterer (0,20
287           288           "received_pressure shape = (128 RX elements, time). Units: m, s,
289           290           a.u.))
291     print()
292     print("Saved figure:", rel(fig_path))
293     print("Saved data :", rel(npz_path))
294
295     # -----
296     # STEP 4: attenuation at 4 / 8 / 12 MHz

```

```

297 # -----
298 def step4_attenuation(params):
299     p = params["attenuation_plot"]
300     banner("STEP 4: Attenuation vs depth at 4, 8, 12 MHz (calculate_attenuation)")
301     depth = np.linspace(0.0, p["max_depth_m"], 301)
302     fig, axs = plt.subplots(1, 2, figsize=(11, 4))
303     print(f"alpha0 = {config.ALPHA_0} dB/MHz/cm. One-way and two-way (round trip) loss:")
304     print(" f0          depth 20 mm: one-way dB  amp factor | round-trip dB  amp factor")
305     for f in p["frequencies_hz"]:
306         a = ap.calculate_attenuation(depth, f0=f)
307         axs[0].plot(depth * 1e3, -20 * np.log10(a), label=f"{f / 1e6:.0f} MHz")
308         axs[1].plot(depth * 1e3, a ** 2, label=f"{f / 1e6:.0f} MHz")
309         a20 = float(ap.calculate_attenuation(0.020, f0=f))
310         print(f"    {f / 1e6:4.0f} MHz          {-20 * np.log10(a20):6.2f}      {a20:.4f}   |"
311               f"    {-40 * np.log10(a20):6.2f}      {a20 ** 2:.4f}")
312     axs[0].set_title("One-way attenuation (dB)")
313     axs[0].set_xlabel("Depth [mm]"); axs[0].set_ylabel("Loss [dB]")
314     axs[1].set_title("Round-trip amplitude factor (linear, = A_TX x A_RX)")
315     axs[1].set_xlabel("Depth [mm]"); axs[1].set_ylabel("Amplitude factor")
316     for ax in axs:
317         ax.axvline(20, color="grey", ls="--", alpha=0.6)
318         ax.legend(); ax.grid(alpha=0.3)
319     fig.tight_layout()
320     path = os.path.join(OUT_DIR, "attenuation_vs_depth_4_8_12MHz.png")
321     fig.savefig(path, dpi=140)
322     plt.close(fig)
323     print("Saved figure:", rel(path))
324
325
326 def main():
327     t0 = time.time()
328     os.makedirs(OUT_DIR, exist_ok=True)
329     tee = Tee(sys.stdout)
330     sys.stdout = tee
331     try:
332         print("NITK-UsoundSim - Team 3: Acoustic Propagation - demo run")
333         print(f"Python {sys.version.split()[0]}, numpy {np.__version__}, matplotlib {matplotlib.___
334     ↪ version__}")
335         print(f"Input file : {rel(INPUT_FILE)}")
336         print(f"Output dir : {rel(OUT_DIR)}")
337         with open(INPUT_FILE) as fh:
338             params = json.load(fh)
339         n_ok, n_ran = step1_unit_tests()
340         step2_team_demo(params)
341         step3_final_probe(params)
342         step4_attenuation(params)
343         banner("SUMMARY")
344         print(f"Unit tests : {n_ok}/{n_ran} passed")
345         print("Output files in output/:")
346         for name in sorted(os.listdir(OUT_DIR)):
347             if name not in ("console_log.txt", "console_screenshot.png", "unit_tests_full_output.t
348     ↪ xt"):
349                 print("  -", name)
350                 print("  - unit_tests_full_output.txt")
351                 print("  - console_log.txt")
352                 print("  - console_screenshot.png")
353                 print(f"Total run time: {time.time() - t0:.1f} s")
354         finally:
355             sys.stdout = tee.stream
356             log = tee.buffer.getvalue()
357             with open(os.path.join(OUT_DIR, "console_log.txt"), "w") as fh:
358                 fh.write(log)
359             console_to_png(log, os.path.join(OUT_DIR, "console_screenshot.png"),
360                             title="python3 run_team3.py (Team 3: Acoustic Propagation)", width_chars=106)
361             print("Console log and screenshot saved in output/. Done.")
362
363
364 if __name__ == "__main__":
365     main()

```

4 Input

What goes into the run

The team's functions take numbers as arguments (not image files), so the input is a small JSON file: `input/team3_run_parameters.json`. Run 1 uses the team's own `config.py`; the JSON only lists those values and `run_team3.py` checks that they match (the run printed `True`). Run 2 and the attenuation plot take their values from the JSON.

Setting	Run 1: team design	Run 2: final probe (L12-4 model)
Elements	64	128
Pitch	$\lambda/2 = 0.154$ mm	0.30 mm
Centre frequency	5 MHz	8 MHz
Sampling frequency	40 MHz	40 MHz
Speed of sound, attenuation	1540 m/s, 0.5 dB/MHz/cm	same (config.py defaults)
Pulse	3-cycle Gaussian	3-cycle Gaussian
Scatterers (x, z), amp	(0, 10), (0, 20), (0, 30) mm, amp 1.0	(0, 20) mm, amp 1.0
Time axis	built by <code>simulate_acoustic_propagation()</code>	built by <code>run_team3.py</code> : start at -half-width, 15 % margin

Attenuation plot: frequencies 4, 8 and 12 MHz, depth 0 to 60 mm.

`input/team3_run_parameters.json`

```

1  {
2    "about": "Sample input for run_team3.py (Team 3: Acoustic Propagation). Distances in metres, fre
3    ← quencies in Hz, speed in m/s.",
4    "run1_team_design": {
5      "about": "The team's own design. These values are read from code/nitk_usoundsim/config.py; the
6      ← y are listed here only so you can see them. run_team3.py checks that they match config.py.",
7      "n_elements": 64,
8      "f0_hz": 5000000.0,
9      "fs_hz": 40000000.0,
10     "c_m_per_s": 1540.0,
11     "alpha0_db_per_mhz_per_cm": 0.5,
12     "pitch": "lambda/2 = (1540 / 5e6) / 2 = 0.154 mm",
13     "scatterers": [
14       {"x": 0.000, "z": 0.010, "amp": 1.0},
15       {"x": 0.000, "z": 0.020, "amp": 1.0},
16       {"x": 0.000, "z": 0.030, "amp": 1.0}
17     ]
18   },
19   "run2_final_probe": {
20     "about": "The project's final probe (Philips L12-4 model). These values are passed as argument
21     ← s to the team's functions; the team's code is not changed.",
22     "probe_name": "Philips L12-4 (project model)",
23     "n_elements": 128,
24     "pitch_m": 0.0003,
25     "f0_hz": 8000000.0,
26     "fs_hz": 40000000.0,
27     "scatterer": {"x": 0.000, "z": 0.020, "amp": 1.0},
28     "time_margin_fraction": 0.15
29   },
30   "attenuation_plot": {
31     "frequencies_hz": [4000000.0, 8000000.0, 12000000.0],
32     "max_depth_m": 0.06
33   }
34 }
```

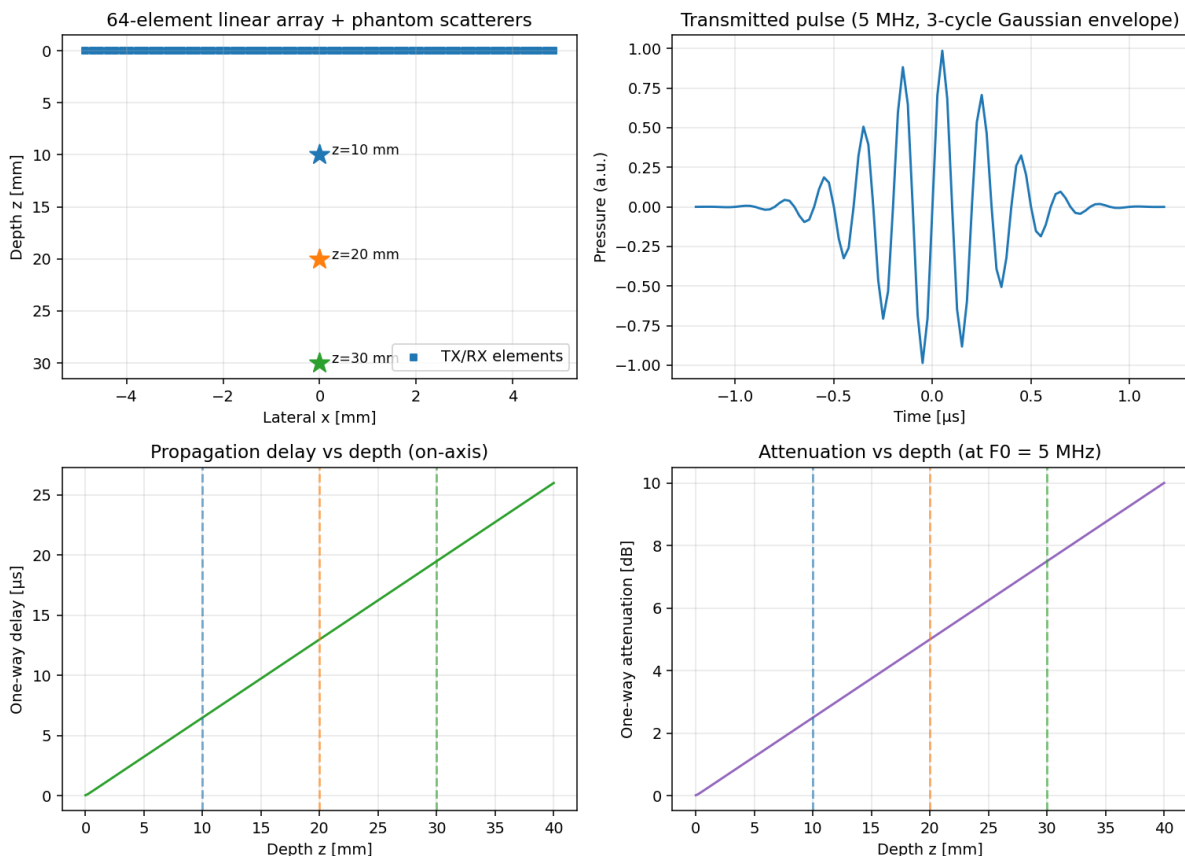
5 Output

Every figure and number produced by run_team3.py

5.1 Unit tests

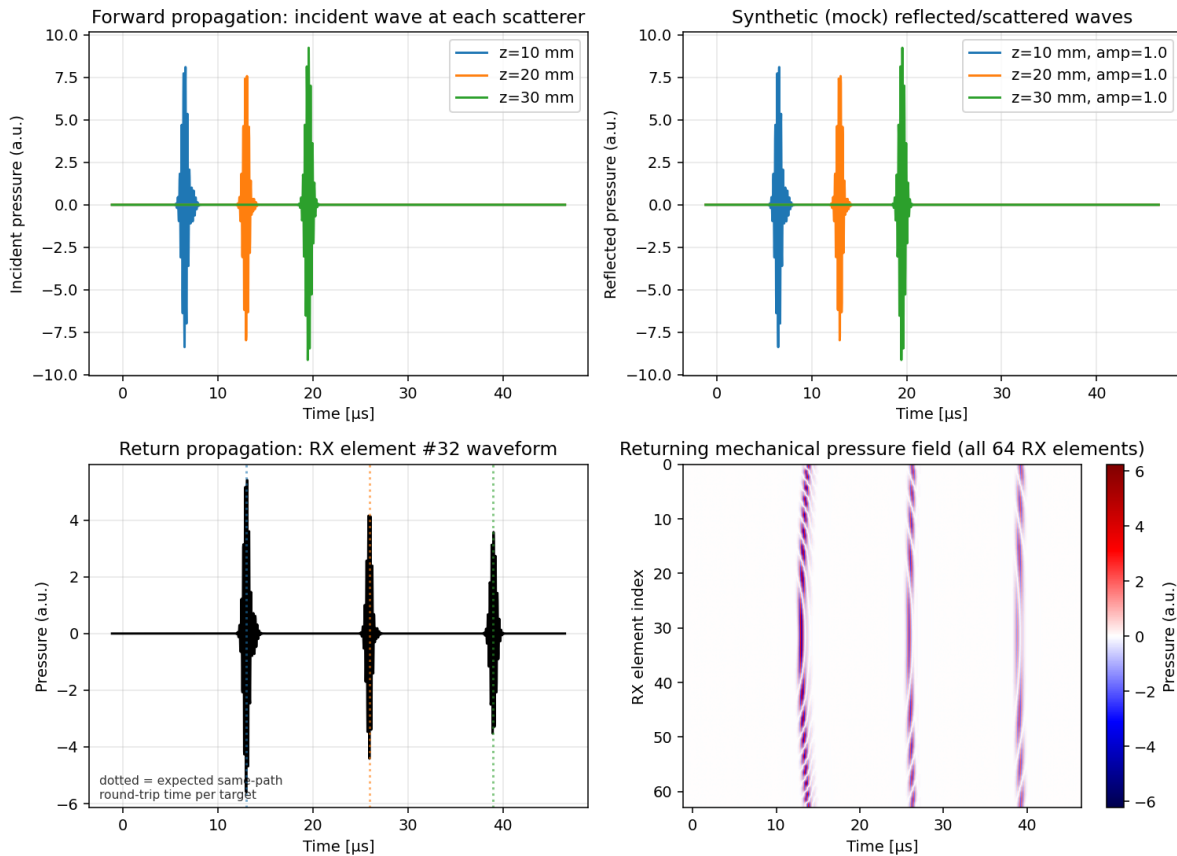
All **22 of 22** tests passed (Ran 22 tests ... 0K). The full unittest text is saved in output/unit_tests_full_output.txt.

5.2 Team demo, Figure 1: geometry, pulse, delay, attenuation



demo_geometry_pulse_delay_attenuation.png. Top left: the 64 elements (9.7 mm wide) and the three scatterers. Top right: the 5 MHz, 3-cycle pulse. Bottom left: one-way delay grows in a straight line with depth (about 6.5 us per 10 mm). Bottom right: one-way loss grows 2.5 dB per 10 mm at 5 MHz (5 dB at 20 mm).

5.3 Team demo, Figure 2: signals through the pipeline

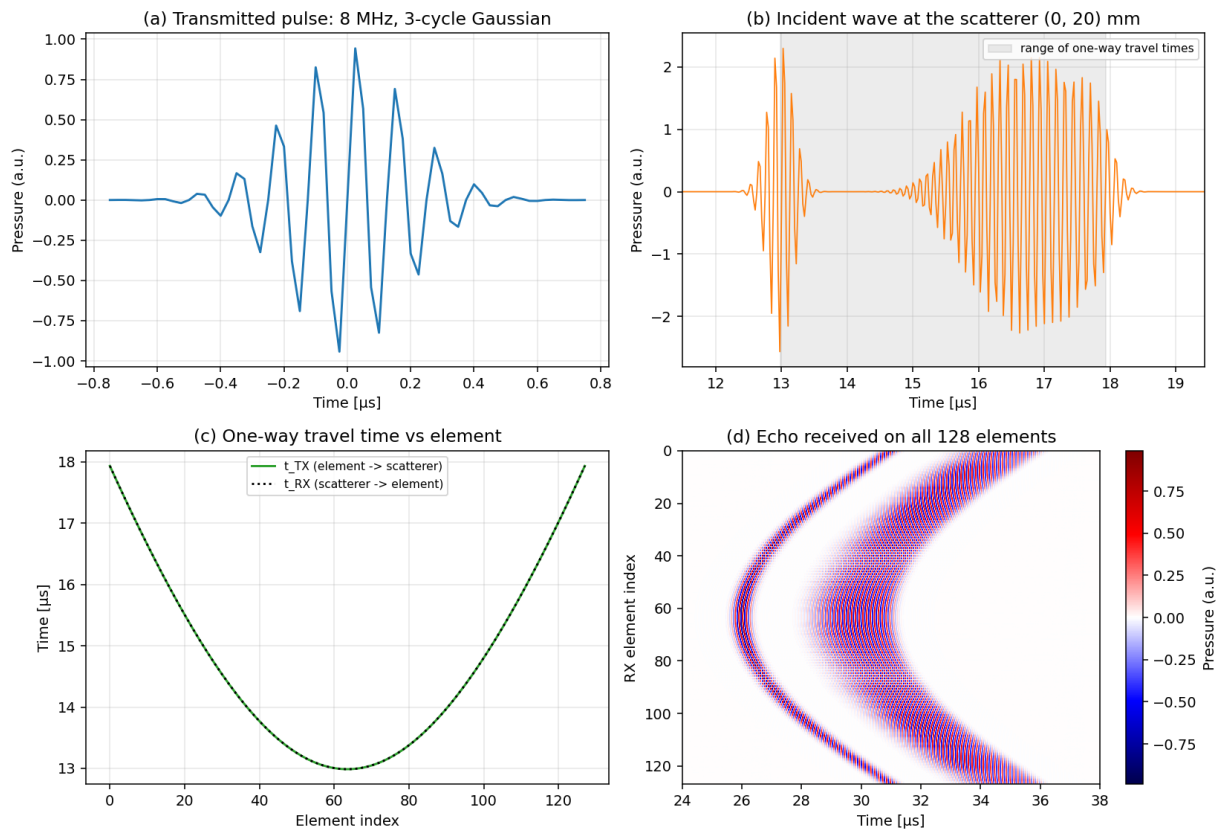


demo_pipeline_signals.png. Top left: incident wave at each scatterer, arriving at about 6.5, 13 and 19.5 μs . Top right: the reflected waves (identical, because $\text{amp} = 1.0$). Bottom left: the echo on element #32 lines up with the dotted expected round-trip times 12.99, 25.97 and 38.96 μs . Bottom right: all 64 elements; each echo is a slightly curved band because side elements are a little further away.

The two .npz files (part1_forward_propagation_outputs.npz and part2_return_propagation_outputs.npz) were compared with the copies the team saved earlier: same keys, and all numbers agree to within $2\text{e-}15$ (floating-point rounding only).

5.4 Final probe run (Philips L12-4 model)

Final probe run: Philips L12-4 model, point scatterer at (0, 20) mm



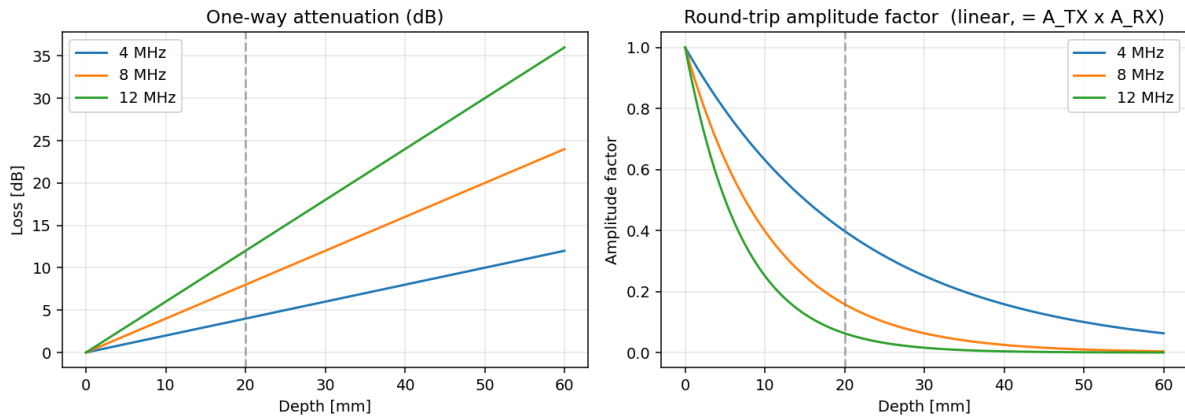
L12-4_final_probe_propagation.png. (a) the 8 MHz pulse (61 samples, ± 0.75 μs). (b) incident wave at (0, 20) mm: a strong burst at about 13 μs from the centre elements, then a longer train up to 17.94 μs from the outer elements (grey band = range of one-way times). (c) one-way time vs element: a U-shape from 12.99 μs (centre) to 17.94 μs (edges); t_{TX} and t_{RX} are identical. (d) echo on all 128 elements: every element receives the whole incident wave, delayed by its own t_{RX} , so the pattern is curved (earliest at the centre, 25.97 μs).

Quantity (from the console)	Value
Aperture / element x range	38.10 mm / -19.05 to 19.05 mm
One-way distance	20.001 to 27.621 mm
One-way time	12.99 to 17.94 μs
Round trip time	25.97 μs (centre) to 35.87 μs (edge)
One-way amplitude factor at 8 MHz	0.3981 (8.00 dB) to 0.2803 (11.05 dB)
Incident wave peak	2.565 a.u. at 12.97 μs
Received pressure array	(128, 1711) = elements x time samples
Peak on element #64 / #0	0.802 a.u. at 26.02 μs / 0.563 a.u. at 30.90 μs

Why is the incident wave so long?

No transmit focusing is used ($\tau_{tx} = 0$), so pulses from the 128 elements reach the scatterer at different times (12.99 to 17.94 μs). Near the centre the times are almost equal, so those pulses add up into one strong burst. Towards the edges each element arrives a bit later. A transmit beamformer (Team 2) would add delays τ_{tx} so all pulses arrive together; `forward_propagation()` already has the τ_{tx} argument for this.

5.5 Attenuation at 4, 8 and 12 MHz



attenuation_vs_depth_4_8_12MHz.png. Left: one-way loss in dB grows linearly with depth and with frequency. Right: round-trip amplitude factor ($A_{TX} \times A_{RX}$). At 20 mm (dashed line): 0.398 at 4 MHz, 0.159 at 8 MHz, 0.063 at 12 MHz. Higher frequency gives finer detail but less depth.

5.6 Console output

output/console_log.txt (exact text printed by python3 run_team3.py)

```
NITK-UsoundSim - Team 3: Acoustic Propagation - demo run
Python 3.12.4, numpy 1.26.4, matplotlib 3.8.3
Input file : input/team3_run_parameters.json
Output dir : output/

=====
STEP 1: Team 3 unit tests (test_acoustic_propagation.py)
=====
Command used: python -m unittest nitk_usoundsim.test_acoustic_propagation -v
(run inside the code/ folder)

ok    TestArrayGeometry.test_array_centred_on_zero
ok    TestArrayGeometry.test_element_count
ok    TestArrayGeometry.test_element_spacing_equals_pitch
ok    TestArrayGeometry.test_wavelength_and_pitch_relationship
ok    TestAttenuation.test_attenuation_at_20mm_is_5dB
ok    TestAttenuation.test_attenuation_is_bounded_between_0_and_1
ok    TestAttenuation.test_attenuation_matches_manual_db_formula
ok    TestAttenuation.test_deeper_targets_experience_more_attenuation
ok    TestDistancesAndDelays.test_deeper_targets_have_larger_tx_delay
ok    TestDistancesAndDelays.test_offaxis_target_gives_varying_rx_delays_across_elements
ok    TestDistancesAndDelays.test_rx_distance_matches_manual_formula
ok    TestDistancesAndDelays.test_rx_propagation_delay
ok    TestDistancesAndDelays.test_tx_distance_matches_manual_formula
ok    TestDistancesAndDelays.test_tx_distance_on_axis_center_equals_depth
ok    TestDistancesAndDelays.test_tx_propagation_delay
ok    TestFullPipeline.test_deeper_phantom_points_have_larger_min_delay
ok    TestFullPipeline.test_multiple_scatterer_linear_superposition
ok    TestFullPipeline.test_output_dimensions
ok    TestFullPipeline.test_output_is_real_valued_mechanical_pressure
ok    TestFullPipeline.test_round_trip_delay_for_20mm_phantom_point
ok    TestPulse.test_pulse_centered_near_zero
ok    TestPulse.test_pulse_sampled_at_fs
Ran 22 tests in 0.025s
OK
(full unittest text saved in output/unit_tests_full_output.txt)

RESULT: 22 of 22 tests passed -> ALL PASSED

=====
STEP 2: Team demo - 64 elements, 5 MHz, lambda/2 pitch (config.py)
=====
Input check: values in input JSON match config.py -> True
Elements      : 64
Centre freq   : 5.0 MHz
```

```

Sampling freq : 40.0 MHz
Speed of sound: 1540 m/s
Wavelength    : 0.3080 mm
Pitch (lam/2)  : 0.1540 mm
Attenuation    : 0.5 dB/MHz/cm
Scatterers     : (0, 10) mm, (0, 20) mm, (0, 30) mm

Saved figure: output/demo_geometry_pulse_delay_attenuation.png
Saved figure: output/demo_pipeline_signals.png
Saved PART 1 (forward propagation) data: output/part1_forward_propagation_outputs.npz
Saved PART 2 (return propagation) data: output/part2_return_propagation_outputs.npz

Time axis      : 1910 samples, -1.175 to 46.550 us
PART 1 incident_pressure shape : (3, 1910) (scatterers, time)
PART 2 received_pressure shape : (64, 1910) (RX elements, time)
Per scatterer (on-axis, round trip = min t_TX + min t_RX):
  z = 10 mm: one-way t = 6.494- 7.217 us | round trip = 12.987 us (2z/c = 12.987) | 2-way amp = 0.562
  z = 20 mm: one-way t = 12.987-13.364 us | round trip = 25.974 us (2z/c = 25.974) | 2-way amp = 0.316
  z = 30 mm: one-way t = 19.481-19.734 us | round trip = 38.961 us (2z/c = 38.961) | 2-way amp = 0.178

=====
STEP 3: Final probe - Philips L12-4 model (128 el, 0.30 mm, 8 MHz)
=====
Done ONLY by passing arguments to the team's functions (code unchanged).
  Elements      : 128 pitch = 0.30 mm aperture = 38.10 mm
  Element x range : -19.05 to 19.05 mm
  Pulse          : 8 MHz, 3 cycles, sigma = 0.1875 us, half-width = 0.7500 us, 61 samples
  Scatterer      : (0, 20) mm, amp = 1.0
  Time axis      : 1711 samples at 40 MHz, -0.750 to 42.000 us
  One-way distance: 20.001 to 27.621 mm
  One-way time    : 12.99 to 17.94 us (centre to edge elements)
  Round trip time : 25.97 us (centre element) to 35.87 us (edge element)
  One-way amplitude factor at 8 MHz: 0.3981 (centre) to 0.2803 (edge) = 8.00 to 11.05 dB
  Incident wave peak |p| = 2.565 a.u. at t = 12.97 us (sum of 128 attenuated pulses)
  Received pressure array shape: (128, 1711) (RX elements, time)
  Element #64: peak |p| = 0.802 a.u. at t = 26.02 us
  Element #0 : peak |p| = 0.563 a.u. at t = 30.90 us

Saved figure: output/L12-4_final_probe_propagation.png
Saved data : output/L12-4_final_probe_outputs.npz

=====
STEP 4: Attenuation vs depth at 4, 8, 12 MHz (calculate_attenuation)
=====
alpha0 = 0.5 dB/MHz/cm. One-way and two-way (round trip) loss:
  f0      depth 20 mm: one-way dB  amp factor | round-trip dB  amp factor
    4 MHz           4.00      0.6310 |      8.00      0.3981
    8 MHz           8.00      0.3981 |     16.00      0.1585
   12 MHz          12.00      0.2512 |     24.00      0.0631
Saved figure: output/attenuation_vs_depth_4_8_12MHz.png

=====
SUMMARY
=====
Unit tests : 22/22 passed
Output files in output/:
- L12-4_final_probe_outputs.npz
- L12-4_final_probe_propagation.png
- attenuation_vs_depth_4_8_12MHz.png
- demo_geometry_pulse_delay_attenuation.png
- demo_pipeline_signals.png
- part1_forward_propagation_outputs.npz
- part2_return_propagation_outputs.npz
- unit_tests_full_output.txt
- console_log.txt
- console_screenshot.png
Total run time: 1.9 s

```

```

● ● ● python3 run_team3.py (Team 3: Acoustic Propagation)

NITK-UsoundSim - Team 3: Acoustic Propagation - demo run
Python 3.12.4, numpy 1.26.4, matplotlib 3.8.3
Input file : input/team3_run_parameters.json
Output dir : output/

=====
STEP 1: Team 3 unit tests (test_acoustic_propagation.py)
=====
Command used: python -m unittest nitk_usoundsim.test_acoustic_propagation -v
(run inside the code/ folder)

ok TestArrayGeometry.test_array_centred_on_zero
ok TestArrayGeometry.test_element_count
ok TestArrayGeometry.test_element_spacing_equals_pitch
ok TestArrayGeometry.test_wavelength_and_pitch_relationship
ok TestAttenuation.test_attenuation_at_20mm_is_5dB
ok TestAttenuation.test_attenuation_is_bounded_between_0_and_1
ok TestAttenuation.test_attenuation_matches_manual_db_formula
ok TestAttenuation.test_deeper_targets_experience_more_attenuation
ok TestDistancesAndDelays.test_deeper_targets_have_larger_tx_delay
ok TestDistancesAndDelays.test_offaxis_target_gives_varying_rx_delays_across_elements
ok TestDistancesAndDelays.test_rx_distance_matches_manual_formula
ok TestDistancesAndDelays.test_rx_propagation_delay
ok TestDistancesAndDelays.test_tx_distance_matches_manual_formula
ok TestDistancesAndDelays.test_tx_distance_on_axis_center_equals_depth
ok TestDistancesAndDelays.test_tx_propagation_delay
ok TestFullPipeline.test_deeper_phantom_points_have_larger_min_delay
ok TestFullPipeline.test_multiple_scatterer_linear_superposition
ok TestFullPipeline.test_output_dimensions
ok TestFullPipeline.test_output_is_real_valued_mechanical_pressure
ok TestFullPipeline.test_round_trip_delay_for_20mm_phantom_point
ok TestPulse.test_pulse_centered_near_zero
ok TestPulse.test_pulse_sampled_at_fs
Ran 22 tests in 0.025s
OK
(full unittest text saved in output/unit_tests_full_output.txt)

RESULT: 22 of 22 tests passed -> ALL PASSED

=====
STEP 2: Team demo - 64 elements, 5 MHz, lambda/2 pitch (config.py)
=====
Input check: values in input JSON match config.py -> True
Elements : 64
Centre freq : 5.0 MHz
Sampling freq : 40.0 MHz
Speed of sound: 1540 m/s
Wavelength : 0.3080 mm
Pitch (lam/2) : 0.1540 mm
Attenuation : 0.5 dB/MHz/cm
Scatterers : (0, 10) mm, (0, 20) mm, (0, 30) mm

Saved figure: output/demo_geometry_pulse_delay_attenuation.png
Saved figure: output/demo_pipeline_signals.png
Saved PART 1 (forward propagation) data: output/part1_forward_propagation_outputs.npz
Saved PART 2 (return propagation) data: output/part2_return_propagation_outputs.npz

Time axis : 1910 samples, -1.175 to 46.550 us
PART 1 incident pressure shape : (3, 1910) (scatterers, time)
PART 2 received pressure shape : (64, 1910) (RX elements, time)
Per scatterer (on-axis, round trip = min t_TX + min t_RX):
z = 10 mm: one-way t = 6.494- 7.217 us | round trip = 12.987 us (2z/c = 12.987) | 2-way amp = 0.562
z = 20 mm: one-way t = 12.987-13.364 us | round trip = 25.974 us (2z/c = 25.974) | 2-way amp = 0.316
z = 30 mm: one-way t = 19.481-19.734 us | round trip = 38.961 us (2z/c = 38.961) | 2-way amp = 0.178

=====
STEP 3: Final probe - Philips L12-4 model (128 el, 0.30 mm, 8 MHz)
=====
Done ONLY by passing arguments to the team's functions (code unchanged).
Elements : 128 pitch = 0.30 mm aperture = 38.10 mm
Element x range : -19.05 to 19.05 mm
Pulse : 8 MHz, 3 cycles, sigma = 0.1875 us, half-width = 0.7500 us, 61 samples
Scatterer : (0, 20) mm, amp = 1.0
Time axis : 1711 samples at 40 MHz, -0.750 to 42.000 us
One-way distance: 20.001 to 27.621 mm
One-way time : 12.99 to 17.94 us (centre to edge elements)
Round trip time : 25.97 us (centre element) to 35.87 us (edge element)
One-way amplitude factor at 8 MHz: 0.3981 (centre) to 0.2803 (edge) = 8.00 to 11.05 dB
Incident wave peak |p| = 2.565 a.u. at t = 12.97 us (sum of 128 attenuated pulses)
Received pressure array shape: (128, 1711) (RX elements, time)
Element #64: peak |p| = 0.802 a.u. at t = 26.02 us
Element #0 : peak |p| = 0.563 a.u. at t = 30.90 us

Saved figure: output/L12-4_final_probe_propagation.png
Saved data : output/L12-4_final_probe_outputs.npz

=====
STEP 4: Attenuation vs depth at 4, 8, 12 MHz (calculate attenuation)
=====
alpha0 = 0.5 dB/MHz/cm. One-way and two-way (round trip) loss:
f0 depth 20 mm: one-way dB amp factor | round-trip dB amp factor
4 MHz 4.00 0.6310 | 8.00 0.3981
8 MHz 8.00 0.3981 | 16.00 0.1585
12 MHz 12.00 0.2512 | 24.00 0.0631
Saved figure: output/attenuation_vs_depth_4_8_12MHz.png

=====
SUMMARY
=====
Unit tests : 22/22 passed
Output files in output/:
- L12-4_final_probe_outputs.npz
- L12-4_final_probe_propagation.png
- attenuation_vs_depth_4_8_12MHz.png
- demo_geometry_pulse_delay_attenuation.png
- demo_pipeline_signals.png
- part1_forward_propagation_outputs.npz
- part2_return_propagation_outputs.npz
- unit_tests_full_output.txt
- console_log.txt
- console_screenshot.png
Total run time: 1.9 s

```

output/console_screenshot.png: the same console text as an image.

6 How to run

Step-by-step, on any computer

1. Install Python 3 (3.9 or newer) from python.org. On Windows, tick *Add Python to PATH* during installation.
2. Copy the folder `Team_3_Acoustic_Propagation` anywhere you like.
3. Open a terminal in that folder (macOS: right-click the folder in Finder, *New Terminal at Folder*; Windows: open the folder, type `cmd` in the address bar, press Enter).
4. Install the packages: `pip install -r requirements.txt` (macOS/Linux may need `pip3` or `python3 -m pip`).
5. Run: `python3 run_team3.py` (macOS/Linux) or `python run_team3.py` (Windows).
6. Open the `output/` folder to see the plots, data, `console_log.txt` and `console_screenshot.png`.

Expected runtime: measured 2 to 5 seconds in total (the script itself reports about 1.5 to 2 s) on the test Mac.

Running the team's own commands

The team's docstrings say to run from the `usound_sim` folder. In this package that folder is `code/` (it contains `nitk_usoundsim/`):

Optional, from the code/ folder

```
cd code
python3 -m unittest nitk_usoundsim.test_acoustic_propagation -v
python3 -m nitk_usoundsim.demo_acoustic_propagation
```

Note: run this way, the demo overwrites its two PNG and two .npz files inside `code/nitk_usoundsim/`. `run_team3.py` avoids that by writing to `output/`.

Troubleshooting

Problem	Fix
<code>python3 / python: command not found</code>	Install Python, or on Windows try <code>py run_team3.py</code> .
<code>ModuleNotFoundError: numpy or matplotlib</code>	Run <code>python3 -m pip install -r requirements.txt</code> with the same Python.
<code>ModuleNotFoundError: nitk_usoundsim</code>	Run the command from inside the <code>Team_3_Acoustic_Propagation</code> folder and keep the <code>code/</code> folder next to <code>run_team3.py</code> .
No plots appear on screen	That is normal: plots are saved as PNG files in <code>output/</code> , not shown in a window.
Permission error writing <code>output/</code>	Copy the folder to a place where you can write (e.g. Desktop or Documents).

7 Check yourself

Viva-style questions with short answers

Q1. How long does sound take to reach a scatterer 20 mm straight below the probe, and to come back?

Answer: $t = d / c = 0.020 / 1540 = 12.99 \text{ us}$ one way, 25.97 us round trip.

Q2. Why does the attenuation formula use $10^{(-dB/20)}$ and not $10^{(-dB/10)}$?

Answer: Decibels are defined on power, and power goes with pressure squared. The code works with pressure amplitude, so it uses 20.

Q3. Why do edge elements of the 128-element probe receive the echo later than the centre element?

Answer: They are further from the scatterer (27.62 mm vs 20.00 mm by Pythagoras), so the one-way time grows from 12.99 us to 17.94 us.

Q4. Why does `forward_propagation` use the pulse formula, but `return_propagation` uses `np.interp`?

Answer: The transmitted pulse is a formula, so it can be evaluated exactly at any delayed time. The reflected wave is only known as samples, so values between samples are found by linear interpolation.

Q5. Is 40 MHz sampling enough for an 8 MHz pulse?

Answer: Yes. Nyquist needs more than twice the highest frequency; the pulse energy is well below 20 MHz, and there are 5 samples per 8 MHz period.

Q6. Name two simplifications of this model and what would be needed to remove them.

Answer: E.g. point elements (add element width/directivity), no 1/r spreading (divide by distance), constant c (allow different tissues), mock tissue (replace with Team 4's real tissue interaction).